

PARTE 3 · DISEÑO DE BASES DE DATOS

De cero absoluto a nivel experto



CLIENTES

id	nombre	email
1	Ana	ana@...
2	Luis	luis@...

```
SELECT nombre
FROM clientes
WHERE id = 1;

-- Tu 1ª consulta
```

BASES DE DATOS

MANUAL COMPLETO · SQL · POSTGRESQL · VPS · NUBE

Diseña · Crea · Consulta · Protege · Conecta · Despliega · Mantén · Escala

Contenido de esta parte

Antes de empezar

3.1 Entidades y atributos

3.2 Relaciones y cardinalidad

3.3 Tablas intermedias

3.4 Claves primarias y claves extranjeras

3.5 Restricciones: únicos, obligatorios, nulos y comprobaciones

3.6 Diagramas entidad-relación

3.7 Del diagrama a las tablas

3.8 Convenciones: nombres, fechas y borrado lógico

Resumen de la Parte 3

Glosario de la Parte 3

Mini examen de la Parte 3

Parte 3. Diseño de bases de datos

Antes de empezar

En la Parte 2 aprendiste a analizar una idea y a escribir una lista de entidades, atributos, relaciones y reglas. En esta parte conviertes ese análisis en un **diseño formal**: un diagrama que muestra las tablas y sus conexiones, y las sentencias de SQL que crean esas tablas en un PostgreSQL real.

Al terminar serás capaz de:

- distinguir una relación uno a uno, uno a muchos y muchos a muchos, y crearlas en SQL;
- resolver una relación muchos a muchos con una tabla intermedia;
- elegir una clave primaria (identidad numérica, natural o UUID) y declarar claves extranjeras con su comportamiento al borrar;
- proteger tus datos con restricciones (obligatorio, único, comprobaciones y valores por defecto);
- leer y dibujar un diagrama entidad-relación, y generarlo desde una base de datos real;
- pasar un diagrama a un script SQL siguiendo reglas fijas y ejecutarlo;
- aplicar convenciones de nombres, fechas de creación y borrado lógico.

Cómo se organiza cada punto

Cada punto sigue estos pasos, siempre en el mismo orden:

1. **¿Qué es?, ¿Para qué sirve?, ¿Por qué lo necesito? y ¿Cómo funciona?:** la teoría, explicada con palabras sencillas.
2. **Paso a paso:** lo que debes hacer, en orden, y lo que deberías ver en cada paso.
3. **Código y resultado:** el código completo, escrito en texto para que lo copies, seguido de **la captura real de su ejecución** y de una explicación de lo que muestra, línea a línea.
4. **Ejemplo sencillo y ejemplo real.**
5. **Profundizando:** variantes, casos límite y detalles que usan los profesionales.
6. **Ejercicio, solución, error habitual, buena práctica y comprobación.**

Cómo ejecutar los ejemplos

Todo el SQL de esta parte se ha ejecutado en **PostgreSQL 16.14**. Las capturas con fondo oscuro son la **salida real de `psql`**, el cliente de terminal de PostgreSQL, tal y como aparece en pantalla (los mensajes están en inglés porque así los emite el servidor). Los diagramas de tablas con claves se generan leyendo la estructura real de la base de datos, no están dibujados a mano. Las figuras que son simples ilustraciones lo dicen en su texto.

Para ejecutar tú los ejemplos necesitas PostgreSQL instalado (se explica en la Parte 6) y conectarte con `psql`. Puedes leer esta parte sin ejecutar nada y volver más tarde a practicar.

Dentro de `psql`, cada sentencia termina en punto y coma (`;`); las órdenes que empiezan por barra invertida (`\d`, `\dt`) son comandos propios de `psql` y no llevan punto y coma.

La base de datos principal de esta parte se llama `tienda` y se construye poco a poco. Para las pruebas que podrían ensuciarla se usa otra base de datos llamada `ensayo`.

3.1 Entidades y atributos

¿Qué es?

En el diseño, cada **entidad** del análisis se convierte en una **tabla**, y cada **atributo** se convierte en una **columna** con un tipo de dato. Es el paso de las ideas a la estructura: de «cliente: nombre, email, teléfono, fecha de alta» a una tabla `clientes` con cinco columnas.

Además del nombre y el tipo, para cada columna se decide si es obligatoria, si es única y si tiene un valor por defecto (lo viste en el punto 2.3). Esas decisiones se guardan en la propia tabla y PostgreSQL las hace cumplir.

¿Para qué sirve?

Para dejar de hablar de «cosas» y empezar a hablar de estructuras que un gestor entiende. Una vez que la tabla existe, el gestor sabe qué datos admite y cuáles rechaza.

¿Por qué lo necesito?

Un buen diseño de tablas hace fáciles las consultas y difíciles los errores. Si decides bien el tipo y las reglas de cada columna, muchos datos erróneos ni siquiera llegarán a guardarse.

¿Cómo funciona?

Para cada entidad:

1. Escribe el nombre de la tabla en plural y en minúsculas (`clientes`).
2. Añade una columna identificadora (normalmente `id`). Cada fila necesita una.
3. Convierte cada atributo en una columna y elige su tipo (punto 1.5 de la Parte 1).
4. Decide, columna a columna, si es obligatoria, única o tiene valor por defecto.
5. Comprueba que cada columna guarda **un solo dato** y que la tabla describe **una sola cosa**.

Paso a paso: crear la tabla `clientes`

1. **Conéctate a la base de datos** `tienda` con `psql` . Verás el prompt `tienda=#` , que significa «estoy conectado a `tienda` y espero una orden».
2. **Escribe la sentencia** `CREATE TABLE` del código de abajo. Como ocupa varias líneas, `psql` cambia el prompt a `tienda-#` mientras la sentencia no termina con `;` .
3. **Pulsa Intro tras el `;`** . Si todo va bien, `psql` responde `CREATE TABLE` : la tabla existe.
4. **Verifica con `\d clientes`** . Debes ver las cinco columnas, sus tipos, cuáles son obligatorias y los índices.
5. **Inserta datos de prueba** con `INSERT` y comprueba con `SELECT * FROM clientes;` . Verás tres filas; las columnas `id` y `creado_en` las habrá rellenado PostgreSQL.

Código y resultado

Lectura línea a línea de la sentencia `CREATE TABLE` :

- `CREATE TABLE clientes (` : «crea una tabla llamada `clientes` »; lo que va entre paréntesis son sus columnas.
- `id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY` : columna `id` de número entero, que PostgreSQL rellena automáticamente (1, 2, 3...) y que es la clave primaria.
- `nombre text NOT NULL` : texto obligatorio.
- `email text NOT NULL UNIQUE` : texto obligatorio y que no puede repetirse.
- `telefono text` : texto opcional (al no poner `NOT NULL` , admite `NULL`).
- `creado_en timestamptz NOT NULL DEFAULT now()` : fecha y hora con zona horaria, obligatoria; si nadie la indica, se usa el momento actual.

```
CREATE TABLE clientes (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre      text NOT NULL,
  email       text NOT NULL UNIQUE,
  telefono    text,
  creado_en   timestamptz NOT NULL DEFAULT now()
);

\d clientes
```

psql — base de datos «tienda»

```
tienda=# CREATE TABLE clientes (
tienda-#   id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
tienda-#   nombre     text NOT NULL,
tienda-#   email      text NOT NULL UNIQUE,
tienda-#   telefono   text,
tienda-#   creado_en  timestamptz NOT NULL DEFAULT now()
tienda-# );
tienda-# CREATE TABLE

tienda=# \d clientes

          Table "public.clientes"
  Column |          Type          | Collation | Nullable |          Default
-----+-----+-----+-----+-----
id       | integer               |           | not null | generated always as identity
nombre   | text                  |           | not null |
email    | text                  |           | not null |
telefono | text                  |           |         |
creado_en | timestamp with time zone |           | not null | now()
Indexes:
    "clientes_pkey" PRIMARY KEY, btree (id)
    "clientes_email_key" UNIQUE CONSTRAINT, btree (email)
```

Figura 3.1-a. Captura real: creación de la tabla `clientes` y descripción de su estructura con `\d`.

Qué ves en la imagen. Tras `CREATE TABLE` , el comando `\d clientes` (la «d» viene de *describe*) muestra:

- **Column:** el nombre de cada columna.

- **Type:** su tipo. `integer` es entero; `text`, texto; `timestamp with time zone` es lo que escribimos abreviado como `timestamptz`.
- **Nullable:** `not null` si la columna es obligatoria; vacío si admite NULL (como `telefono`).
- **Default:** el valor por defecto. `id` aparece como `generated always as identity` y `creado_en` como `now()`.
- **Indexes:** los índices que PostgreSQL ha creado por sí mismo para vigilar las claves: la clave primaria (`clientes_pkey`) y el email único (`clientes_email_key`).

Ahora insertamos tres clientes. Fíjate en que no indicamos ni el `id` ni `creado_en`:

```
INSERT INTO clientes (nombre, email, telefono) VALUES
('Ana García', 'ana@ejemplo.com', '600111222'),
('Luis Pérez', 'luis@ejemplo.com', NULL),
('Marta Ruiz', 'marta@ejemplo.com', '600333444');

SELECT * FROM clientes ORDER BY id;
```

```
psql — base de datos «tienda»

tienda=# INSERT INTO clientes (nombre, email, telefono) VALUES
tienda=# ('Ana García', 'ana@ejemplo.com', '600111222'),
tienda=# ('Luis Pérez', 'luis@ejemplo.com', NULL),
tienda=# ('Marta Ruiz', 'marta@ejemplo.com', '600333444');
INSERT 0 3

tienda=# SELECT * FROM clientes ORDER BY id;
 id | nombre | email | telefono | creado_en
-----+-----+-----+-----+-----
  1 | Ana García | ana@ejemplo.com | 600111222 | 2026-10-03 08:11:41.619902+00
  2 | Luis Pérez | luis@ejemplo.com |          | 2026-10-03 08:11:41.619902+00
  3 | Marta Ruiz | marta@ejemplo.com | 600333444 | 2026-10-03 08:11:41.619902+00
(3 rows)
```

Figura 3.1-b. Captura real: inserción de tres clientes y lectura de la tabla.

Qué ves en la imagen. La salida `INSERT 0 3` significa «se han insertado 3 filas» (el 0 es un dato histórico que ahora no nos importa). El `SELECT * FROM clientes` devuelve las tres filas: `id` va de 1 a 3 sin que lo escribiéramos, `creado_en` tiene la fecha y hora del momento exacto de la inserción y el teléfono de Luis aparece vacío, porque es NULL.



Figura 3.1-c. Ilustración: una entidad del análisis se convierte en una tabla.

Qué ves en la imagen. A la izquierda, la tarjeta de la entidad tal y como salió del análisis; a la derecha, la tabla que la representa: una columna por atributo, más el `id`.

Ejemplo sencillo

La entidad cliente tiene cuatro atributos (nombre, email, teléfono, fecha de alta). La tabla tiene esas cuatro columnas más el `id`. El nombre y el email son obligatorios; el email, además, único; el teléfono, opcional; y la fecha de alta se rellena sola.

Ejemplo real

En una tienda online, la tabla `clientes` suele tener también una dirección, un identificador fiscal y una fecha de último acceso. Al crecer el proyecto, se añaden columnas con `ALTER TABLE` (lo verás en el punto 3.8), pero el diseño inicial sigue siendo exactamente este.

Profundizando

Elegir el tipo de texto. En PostgreSQL, `text` admite cualquier longitud y es la opción más cómoda. También existe `varchar(n)`, que limita la longitud a `n` caracteres. Rara vez hace falta limitar: si necesitas una longitud máxima, suele ser mejor una restricción `CHECK` explícita.

Elegir el tipo de número. `integer` guarda enteros hasta aproximadamente 2.100 millones; si esperas más filas, existe `bigint`. Para dinero usa siempre `numeric` (exacto) y no tipos de coma flotante (`real`, `double precision`), que son aproximados. La demostración siguiente lo muestra con una suma tan simple como $0,1 + 0,2$:

```
SELECT 0.1::double precision + 0.2::double precision AS coma_flotante,
       0.1::numeric + 0.2::numeric AS exacto;

SELECT '2026-03-02 10:00:00+01'::timestampz AT TIME ZONE 'UTC' AS en_utc,
       '2026-03-02 10:00:00+01'::timestampz AT TIME ZONE 'Europe/Madrid' AS
en_madrid;
```

psql — base de datos «ensayo»

```
ensayo=# SELECT 0.1::double precision + 0.2::double precision AS coma_flotante,
ensayo-#         0.1::numeric + 0.2::numeric AS exacto;
coma_flotante | exacto
-----+-----
0.30000000000000004 | 0.3
(1 row)

ensayo=# SELECT '2026-03-02 10:00:00+01'::timestampz AT TIME ZONE 'UTC' AS en_utc,
ensayo-#         '2026-03-02 10:00:00+01'::timestampz AT TIME ZONE 'Europe/Madrid' AS en_madrid;
en_utc | en_madrid
-----+-----
2026-03-02 09:00:00 | 2026-03-02 10:00:00
(1 row)
```

Figura 3.1-d. Captura real: por qué el dinero se guarda con `numeric` y cómo se comporta la zona horaria.

Qué ves en la imagen. Con `double precision` (coma flotante), $0,1 + 0,2$ da `0.30000000000000004`: un error diminuto, pero que en facturas acumula céntimos de más o de menos. Con `numeric` el resultado es exactamente `0.3`. En la segunda consulta, el mismo instante (`10:00` en España, UTC+1) se muestra como `09:00` en UTC y `10:00` en Madrid:

`timestamptz` guarda un **instante** y lo convierte a la zona horaria de quien consulta, y por eso es la mejor opción para fechas con hora.

Fecha con o sin zona horaria. `timestamp` guarda una fecha y hora «tal cual», sin zona; `timestamptz` guarda el instante real. Para registrar cuándo ocurrió algo, usa `timestamptz`.

Ejercicio

1. Escribe el `CREATE TABLE` de una tabla `productos` con: `id` (identidad), `nombre` (texto obligatorio), `precio_eur` (decimal exacto, obligatorio, no negativo), `stock` (entero obligatorio, por defecto 0) y `activo` (verdadero/falso, por defecto verdadero).
2. ¿Qué diferencia hay entre `telefono text` y `telefono text NOT NULL`?
3. En la figura 3.1-a, ¿qué columnas pueden estar vacías?
4. ¿Por qué `precio_eur` es `numeric(10, 2)` y no `double precision`?

Solución

1. Esta es la sentencia (la verás ejecutada en el punto 3.3):

```
CREATE TABLE productos (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre      text NOT NULL,
  precio_eur  numeric(10, 2) NOT NULL CHECK (precio_eur >= 0),
  stock       integer NOT NULL DEFAULT 0,
  activo      boolean NOT NULL DEFAULT true
);
```

1. Sin `NOT NULL`, la columna admite NULL (puede quedarse sin valor). Con `NOT NULL`, toda fila nueva debe traer un teléfono.
2. Solo `telefono`: es la única cuya columna «Nullable» está vacía.
3. Porque `numeric` es exacto (no acumula errores de céntimos) y `(10, 2)` limita a 10 cifras en total, de las cuales 2 son decimales.

Error habitual

Poner varios datos en una misma columna («nombre y apellidos y teléfono» en un único texto). Después no puedes buscar ni ordenar por cada dato por separado. Una columna, un dato.

```
-- Mal: todo mezclado en una sola columna
CREATE TABLE clientes_mal (contacto text);
-- Bien: un dato por columna
CREATE TABLE clientes_bien (nombre text, email text, telefono text);
```

Buena práctica

Antes de crear una tabla, escribe en papel (o en la plantilla de la Parte 2) sus columnas, tipos y reglas. Crear la tabla es después casi mecánico. Y guarda siempre las sentencias en un archivo

`.sql` , no solo en la terminal.

Comprobación

Para una tabla de tu proyecto: ¿cada columna guarda un solo dato? ¿Tiene el tipo adecuado? ¿Has decidido si es obligatoria? Crea la tabla, ejecuta `\d nombre_de_la_tabla` y comprueba que coincide con tu plan. Si algo difiere, corrige el plan o la tabla, pero no los dos a medias.

3.2 Relaciones y cardinalidad

¿Qué es?

Una **relación** es la conexión entre dos entidades. La **cardinalidad** indica **cuántos elementos de un lado pueden estar conectados con cuántos del otro**. Hay tres tipos:

Tipo	Se escribe	Significado	Ejemplo
Uno a uno	1 a 1	Cada elemento de A se conecta con uno de B y viceversa	Usuario y perfil
Uno a muchos	1 a N	Cada elemento de A se conecta con muchos de B, pero cada B con uno solo de A	Autor y libros
Muchos a muchos	N a N	Cada elemento de A se conecta con muchos de B y viceversa	Alumnos y cursos

La «N» significa «muchos» (cualquier cantidad, incluido cero o uno).

¿Para qué sirve?

La cardinalidad decide **cómo se crean las tablas**. Cada tipo se resuelve de una manera distinta en SQL: una relación uno a muchos con una clave extranjera; una uno a uno con una clave extranjera única; una muchos a muchos con una tabla intermedia (punto 3.3).

¿Por qué lo necesito?

Si te equivocas con la cardinalidad, la base de datos queda mal pensada. Si crees que un libro puede tener un solo autor cuando en realidad puede tener varios, tendrás que rehacer tablas con datos ya guardados.

¿Cómo funciona?

Para decidir la cardinalidad, hazte **dos preguntas**, una en cada sentido:

1. ¿Cuántos B puede tener **un** A? (¿cuántos libros puede escribir un autor?)
2. ¿Cuántos A puede tener **un** B? (¿cuántos autores puede tener un libro?)

Con las respuestas: **1 y 1** es uno a uno; **muchos y 1** (o 1 y muchos) es uno a muchos; **muchos y muchos** es muchos a muchos.

Además hay que decidir si la participación es **obligatoria u opcional**: ¿puede existir un libro sin autor? ¿Un autor sin libros? La respuesta se traduce en SQL: la clave extranjera será **NOT NULL** (obligatoria) o admitirá NULL (opcional).

Paso a paso: las tres relaciones en SQL

1. **Uno a uno.** Crea la tabla principal (`usuarios`). Crea la tabla secundaria (`perfiles`) cuya **clave primaria es a la vez clave extranjera** hacia la principal: así no puede haber dos perfiles para el mismo usuario.
2. **Uno a muchos.** Crea la tabla del lado «uno» (`autores`). En la del lado «muchos» (`libros`), añade una columna `autor_id` con `REFERENCES autores (id)` .
3. **Muchos a muchos.** Crea las dos tablas (`alumnos` , `cursos`) y una tercera (`matriculas`) con dos claves extranjeras. Lo detallamos en el punto 3.3.
4. **Comprueba** insertando datos válidos y, a propósito, un dato que rompa la regla: debe ser rechazado.

Código y resultado: uno a uno

```
CREATE TABLE usuarios (
  id    integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email text NOT NULL UNIQUE
);

CREATE TABLE perfiles (
  usuario_id integer PRIMARY KEY REFERENCES usuarios (id),
  biografia  text
);

INSERT INTO usuarios (email) VALUES ('ana@ejemplo.com') RETURNING id;

INSERT INTO perfiles (usuario_id, biografia) VALUES (1, 'Me gusta el senderismo');

INSERT INTO perfiles (usuario_id, biografia) VALUES (1, 'Otro perfil para el mismo usuario');
```

```

psql — base de datos «ensayo»

ensayo=# CREATE TABLE usuarios (
ensayo=#   id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
ensayo=#   email text NOT NULL UNIQUE
ensayo=# );
CREATE TABLE

ensayo=# CREATE TABLE perfiles (
ensayo=#   usuario_id integer PRIMARY KEY REFERENCES usuarios (id),
ensayo=#   biografia text
ensayo=# );
CREATE TABLE

ensayo=# INSERT INTO usuarios (email) VALUES ('ana@ejemplo.com') RETURNING id;
id
----
  1
(1 row)

INSERT 0 1

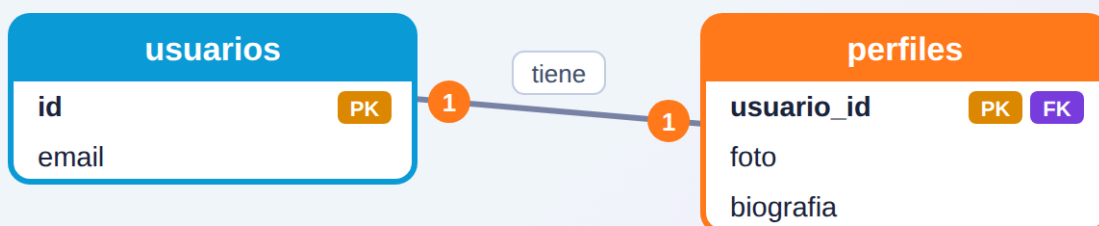
ensayo=# INSERT INTO perfiles (usuario_id, biografia) VALUES (1, 'Me gusta el senderismo');
INSERT 0 1

ensayo=# INSERT INTO perfiles (usuario_id, biografia) VALUES (1, 'Otro perfil para el mismo usuario');
ERROR: duplicate key value violates unique constraint "perfiles_pkey"
DETAIL: Key (usuario_id)=(1) already exists.

```

Figura 3.2-a. Captura real: relación uno a uno. El segundo perfil del mismo usuario es rechazado.

Qué ves en la imagen. `perfiles` usa `usuario_id` como clave primaria y, a la vez, como clave extranjera (`PRIMARY KEY REFERENCES usuarios (id)`). El primer perfil se inserta (`INSERT 0 1`). El segundo, para el mismo usuario, falla con `duplicate key value violates unique constraint "perfiles_pkey"`: la clave primaria impide repetir el `usuario_id`, así que cada usuario tiene como máximo un perfil.



Uno a uno (1 a 1): un usuario tiene un único perfil, y cada perfil es de un único usuario.

Figura 3.2-b. Diagrama de la relación uno a uno.

Qué ves en la imagen. Las etiquetas PK y FK aparecen juntas en `usuario_id`: es la señal de una relación uno a uno resuelta con la clave compartida.

Código y resultado: uno a muchos

```

CREATE TABLE autores (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre text NOT NULL
);

CREATE TABLE libros (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  autor_id integer NOT NULL REFERENCES autores (id),
  titulo  text NOT NULL
);

INSERT INTO autores (nombre) VALUES ('Gabriel García Márquez');

INSERT INTO libros (autor_id, titulo) VALUES
  (1, 'Cien años de soledad'),
  (1, 'El amor en los tiempos del cólera');

SELECT a.nombre, l.titulo
FROM autores a
JOIN libros l ON l.autor_id = a.id;

```



```

psql — base de datos «ensayo»

ensayo=# CREATE TABLE autores (
ensayo-#   id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
ensayo-#   nombre text NOT NULL
ensayo-# );
CREATE TABLE

ensayo=# CREATE TABLE libros (
ensayo-#   id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
ensayo-#   autor_id integer NOT NULL REFERENCES autores (id),
ensayo-#   titulo  text NOT NULL
ensayo-# );
CREATE TABLE

ensayo=# INSERT INTO autores (nombre) VALUES ('Gabriel García Márquez');
INSERT 0 1

ensayo=# INSERT INTO libros (autor_id, titulo) VALUES
ensayo-#   (1, 'Cien años de soledad'),
ensayo-#   (1, 'El amor en los tiempos del cólera');
INSERT 0 2

ensayo=# SELECT a.nombre, l.titulo
ensayo-# FROM autores a
ensayo-# JOIN libros l ON l.autor_id = a.id;
  nombre      |          titulo
-----+-----
Gabriel García Márquez | Cien años de soledad
Gabriel García Márquez | El amor en los tiempos del cólera
(2 rows)

```

Figura 3.2-c. Captura real: relación uno a muchos, con una consulta que une las dos tablas.

Qué ves en la imagen. `libros.autor_id` es la clave extranjera y está en la tabla del lado «muchos». Se inserta un autor y dos libros que apuntan a él (`autor_id = 1`). La última consulta

usa **JOIN** (unir tablas: se estudia en la Parte 5) para mostrar cada libro junto al nombre de su autor. Un solo autor aparece en dos filas, porque tiene dos libros.

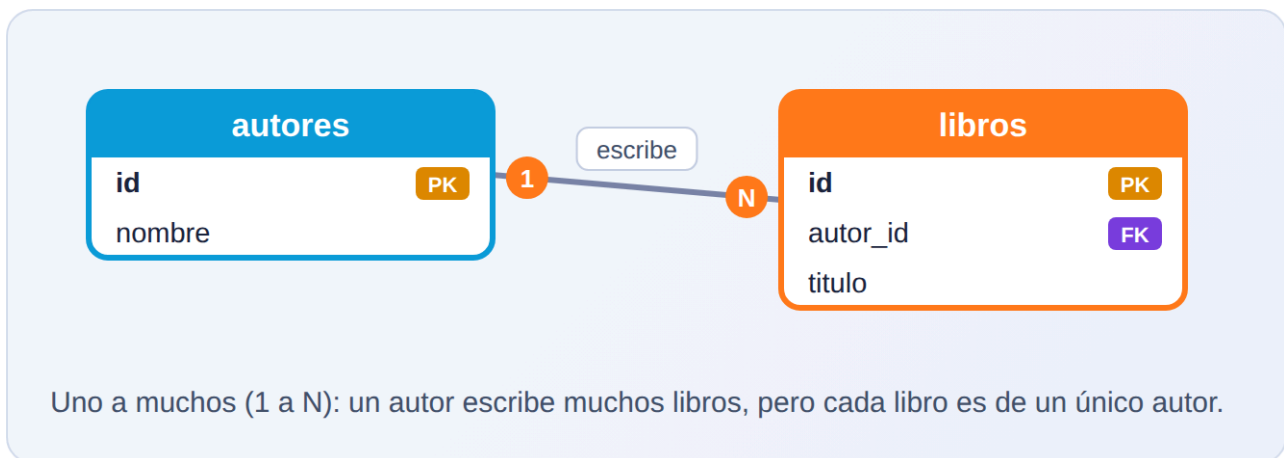


Figura 3.2-d. Diagrama de la relación uno a muchos.

Qué ves en la imagen. El «1» junto a **autores** y la «N» junto a **libros**. La clave extranjera vive siempre en el lado de la «N».

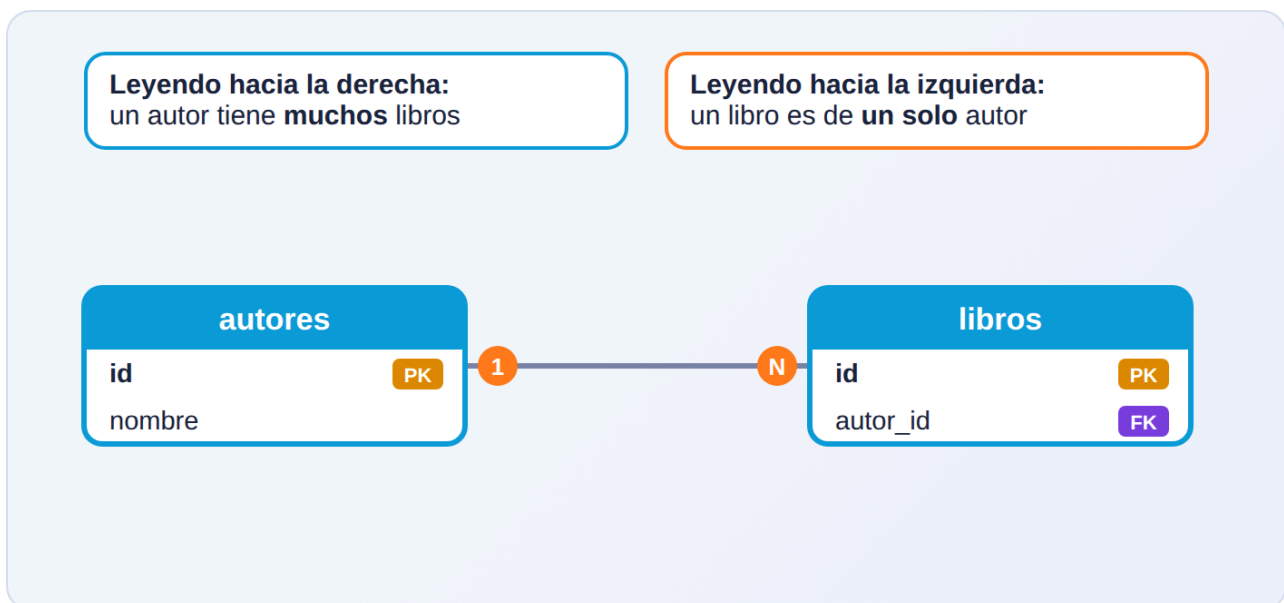


Figura 3.2-e. Cómo se lee una relación: una vez en cada sentido.

Qué ves en la imagen. La misma línea se lee dos veces: hacia la derecha («un autor tiene muchos libros») y hacia la izquierda («un libro es de un solo autor»).

Código y resultado: participación opcional y relación consigo misma

```
INSERT INTO libros (autor_id, titulo) VALUES (NULL, 'Libro sin autor');

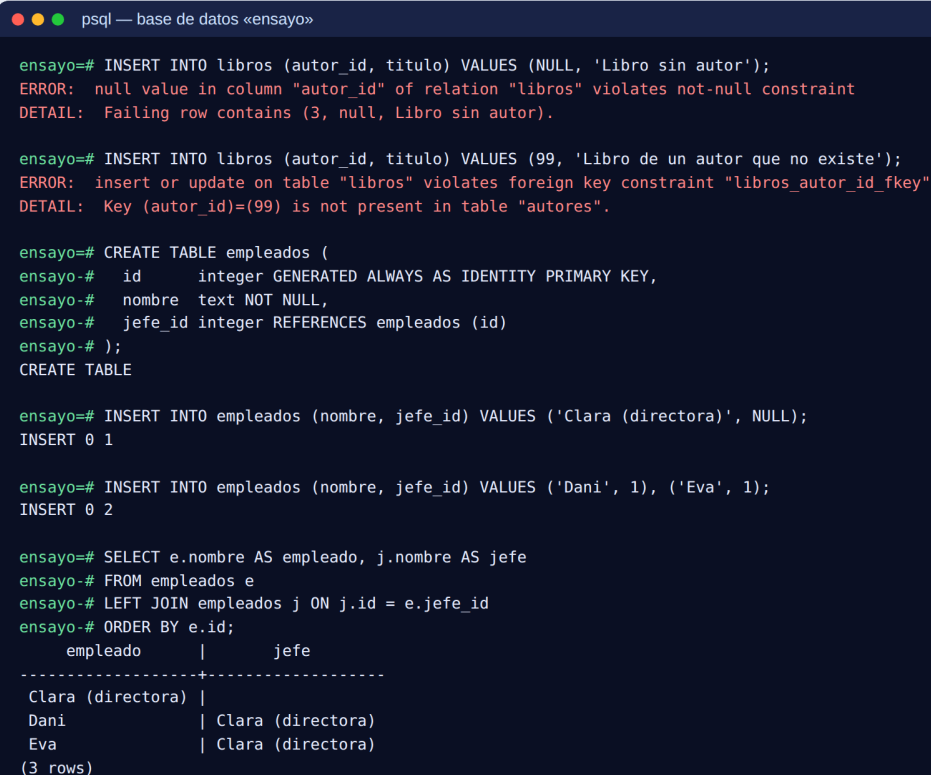
INSERT INTO libros (autor_id, titulo) VALUES (99, 'Libro de un autor que no
existe');

CREATE TABLE empleados (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre  text NOT NULL,
  jefe_id integer REFERENCES empleados (id)
);

INSERT INTO empleados (nombre, jefe_id) VALUES ('Clara (directora)', NULL);

INSERT INTO empleados (nombre, jefe_id) VALUES ('Dani', 1), ('Eva', 1);

SELECT e.nombre AS empleado, j.nombre AS jefe
FROM empleados e
LEFT JOIN empleados j ON j.id = e.jefe_id
ORDER BY e.id;
```



```
psql — base de datos «ensayo»

ensayo=# INSERT INTO libros (autor_id, titulo) VALUES (NULL, 'Libro sin autor');
ERROR: null value in column "autor_id" of relation "libros" violates not-null constraint
DETAIL: Failing row contains (3, null, Libro sin autor).

ensayo=# INSERT INTO libros (autor_id, titulo) VALUES (99, 'Libro de un autor que no existe');
ERROR: insert or update on table "libros" violates foreign key constraint "libros_autor_id_fkey"
DETAIL: Key (autor_id)=(99) is not present in table "autores".

ensayo=# CREATE TABLE empleados (
ensayo-#   id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
ensayo-#   nombre  text NOT NULL,
ensayo-#   jefe_id integer REFERENCES empleados (id)
ensayo-# );
CREATE TABLE

ensayo=# INSERT INTO empleados (nombre, jefe_id) VALUES ('Clara (directora)', NULL);
INSERT 0 1

ensayo=# INSERT INTO empleados (nombre, jefe_id) VALUES ('Dani', 1), ('Eva', 1);
INSERT 0 2

ensayo=# SELECT e.nombre AS empleado, j.nombre AS jefe
ensayo-# FROM empleados e
ensayo-# LEFT JOIN empleados j ON j.id = e.jefe_id
ensayo-# ORDER BY e.id;
 empleado | jefe
-----+-----
 Clara (directora) | 
 Dani       | Clara (directora)
 Eva       | Clara (directora)
(3 rows)
```

Figura 3.2-f. Captura real: participación obligatoria y una relación reflexiva.

Qué ves en la imagen.

1. `libros.autor_id` es `NOT NULL`, así que un libro sin autor es rechazado (`violates not-null constraint`). Fíjate en el primer número de `Failing row contains (3, null, ...)`:

- es el `id` que ya se había reservado (el 3, porque los dos libros anteriores usaron el 1 y el 2).
- Un libro de un autor que no existe (`autor_id` = 99) es rechazado por la clave extranjera.
 - La tabla `empleados` es una **relación reflexiva**: `jefe_id` apunta a la propia tabla. Clara no tiene jefe (`jefe_id` NULL, participación opcional); Dani y Eva sí. La consulta, con `LEFT JOIN`, muestra cada empleado con su jefe y deja vacío el de Clara.

Código y resultado: muchos a muchos

```
CREATE TABLE alumnos (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre text NOT NULL
);

CREATE TABLE cursos (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  titulo text NOT NULL
);

CREATE TABLE matriculas (
  alumno_id integer REFERENCES alumnos (id),
  curso_id  integer REFERENCES cursos (id),
  nota      numeric(4, 2),
  PRIMARY KEY (alumno_id, curso_id)
);

INSERT INTO alumnos (nombre) VALUES ('Ana'), ('Luis');

INSERT INTO cursos (titulo) VALUES ('Inglés'), ('Dibujo');

INSERT INTO matriculas (alumno_id, curso_id, nota) VALUES (1, 1, 8.5), (1, 2,
9), (2, 1, 7);

SELECT a.nombre, c.titulo, m.nota
FROM matriculas m
JOIN alumnos a ON a.id = m.alumno_id
JOIN cursos  c ON c.id = m.curso_id
ORDER BY a.nombre, c.titulo;
```

```

psql — base de datos «ensayo»

ensayo=# CREATE TABLE alumnos (
ensayo=#   id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
ensayo=#   nombre text NOT NULL
ensayo=# );
CREATE TABLE

ensayo=# CREATE TABLE cursos (
ensayo=#   id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
ensayo=#   titulo text NOT NULL
ensayo=# );
CREATE TABLE

ensayo=# CREATE TABLE matriculas (
ensayo=#   alumno_id integer REFERENCES alumnos (id),
ensayo=#   curso_id  integer REFERENCES cursos (id),
ensayo=#   nota      numeric(4, 2),
ensayo=#   PRIMARY KEY (alumno_id, curso_id)
ensayo=# );
CREATE TABLE

ensayo=# INSERT INTO alumnos (nombre) VALUES ('Ana'), ('Luis');
INSERT 0 2

ensayo=# INSERT INTO cursos (titulo) VALUES ('Inglés'), ('Dibujo');
INSERT 0 2

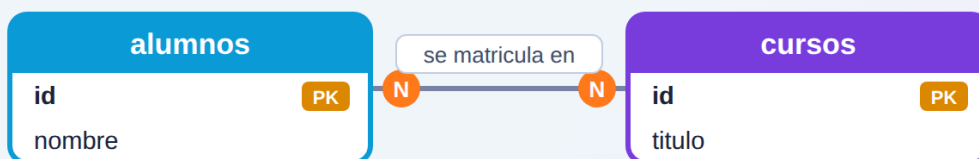
ensayo=# INSERT INTO matriculas (alumno_id, curso_id, nota) VALUES (1, 1, 8.5), (1, 2, 9), (2, 1, 7);
INSERT 0 3

ensayo=# SELECT a.nombre, c.titulo, m.nota
ensayo=# FROM matriculas m
ensayo=# JOIN alumnos a ON a.id = m.alumno_id
ensayo=# JOIN cursos c ON c.id = m.curso_id
ensayo=# ORDER BY a.nombre, c.titulo;
 nombre | titulo | nota
-----+-----+-----
Ana     | Dibujo | 9.00
Ana     | Inglés | 8.50
Luis    | Inglés | 7.00
(3 rows)

```

Figura 3.2-g. Captura real: relación muchos a muchos entre alumnos y cursos.

Qué ves en la imagen. Ana está matriculada en dos cursos y el curso de Inglés tiene dos alumnos: eso es muchos a muchos. La tabla `matriculas` tiene una fila por cada pareja (alumno, curso) y guarda un dato propio de la pareja, la `nota`. La consulta final une las tres tablas para mostrar nombres en lugar de números.



Muchos a muchos (N a N): un alumno se matricula en muchos cursos, y un curso tiene muchos alumnos.

Figura 3.2-h. Diagrama de la relación muchos a muchos (sin resolver).

Qué ves en la imagen. Una «N» en cada extremo. Esta relación no se puede guardar entre dos tablas: requiere la intermedia que acabas de ver.

Ejemplo sencillo

«Un cliente hace muchos pedidos; un pedido es de un único cliente». ¿Cuántos pedidos tiene un cliente? Muchos. ¿Cuántos clientes tiene un pedido? Uno. Resultado: uno a muchos, con la clave extranjera `cliente_id` en `pedidos`.

Ejemplo real

En una academia: «un alumno se matricula en varios cursos y en un curso hay varios alumnos» (muchos a muchos); «un usuario tiene una foto de perfil» (uno a uno); «un curso lo imparte un profesor» (uno a muchos).

Profundizando

Cardinalidad mínima y máxima. La notación completa distingue el mínimo y el máximo de cada lado: «0..1» (cero o uno), «1..1» (exactamente uno), «0..N» (cero o muchos), «1..N» (uno o muchos). En nuestra tienda, un cliente tiene «0..N» pedidos (Luis tiene cero) y un pedido tiene «1..1» cliente. En SQL, el mínimo 0 o 1 se controla con `NOT NULL` en la clave extranjera; el máximo se controla con la propia estructura (claves primarias y únicas).

Uno a uno: ¿por qué dos tablas? Se separa en dos tablas cuando los datos del perfil son opcionales, muy grandes, poco consultados o con permisos distintos. Si siempre se usan juntos y todos son obligatorios, suele ser más simple una sola tabla.

Relaciones con más de dos entidades. A veces una relación une tres entidades a la vez (profesor, asignatura y aula). Se resuelve igual: una tabla intermedia con tres claves extranjeras.

Ejercicio

1. Determina la cardinalidad de: (a) país y ciudades; (b) persona y pasaporte vigente; (c) estudiantes y asignaturas.
2. Escribe el `CREATE TABLE` de `ciudades` para que cada ciudad pertenezca obligatoriamente a un país (suponiendo que la tabla `países` ya existe con una columna `id`).
3. En nuestra tienda, ¿puede existir un cliente sin ningún pedido? ¿Y un pedido sin cliente?

Solución

1. (a) Uno a muchos. (b) Uno a uno. (c) Muchos a muchos.
2. La clave extranjera obligatoria (`NOT NULL`) hacia `países` :

```
CREATE TABLE ciudades (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre  text NOT NULL,
  pais_id integer NOT NULL REFERENCES países (id)
);
```

1. Un cliente sin pedidos sí (Luis, en nuestros datos). Un pedido sin cliente no: `cliente_id` es `NOT NULL` y además debe existir en `clientes`.

Error habitual

Dibujar solo el sentido que se te ocurre. «Un autor escribe libros» parece uno a muchos, pero si olvidas preguntar «¿cuántos autores tiene un libro?», puedes ignorar que hay libros con varios autores, y entonces la relación es muchos a muchos.

```
-- Mal si los libros pueden tener varios autores: autor_id solo admite uno
CREATE TABLE libros_mal (id integer PRIMARY KEY, titulo text, autor_id integer);
```

Buena práctica

Pregunta siempre en los dos sentidos, escribe las dos respuestas y anota si la participación es obligatoria u opcional.

Comprobación

Elige dos entidades de un proyecto tuyo. Escribe dos frases («un A tiene ... B» y «un B tiene ... A»), deduce la cardinalidad, crea las tablas y comprueba con datos reales que se rechaza el caso prohibido.

3.3 Tablas intermedias

¿Qué es?

Una **tabla intermedia** (también llamada tabla de unión, tabla puente o tabla de asociación) es la que se crea para resolver una relación **muchos a muchos**. En lugar de conectar directamente dos tablas, se crea una tercera conectada a las dos, con **una fila por cada pareja**.

¿Para qué sirve?

Permite que un pedido tenga **muchos productos** y que un producto aparezca en **muchos pedidos**, sin repetir datos y sin límites. Además, la tabla intermedia puede guardar datos propios de la relación, como la cantidad o el precio al que se vendió.

¿Por qué lo necesito?

Porque no hay otra forma limpia de guardar una relación muchos a muchos. Las dos soluciones «ingenuas» fallan:

✗ Varios productos en una celda

pedidos (mal)		
id	cliente	productos
101	Ana	Camiseta, Camiseta, Gorra
102	Marta	Mochila

✗ Columnas repetidas

pedidos (mal)			
id	producto1	producto2	producto3
101	Camiseta	Gorra	NULL
103	Camiseta	Mochila	NULL

Ilustración de los dos diseños incorrectos.

Figura 3.3-a. Ilustración: dos soluciones que parecen válidas y no lo son.

Qué ves en la imagen. A la izquierda, varios productos metidos en una sola celda; a la derecha, una columna por producto. En el primer caso no se puede contar bien, ni cambiar un precio, ni asegurar que el producto existe. En el segundo, un pedido de cuatro productos obliga a añadir columnas a toda la tabla, y casi todas las filas se quedan con huecos.

Veamos el primer problema con una consulta real:

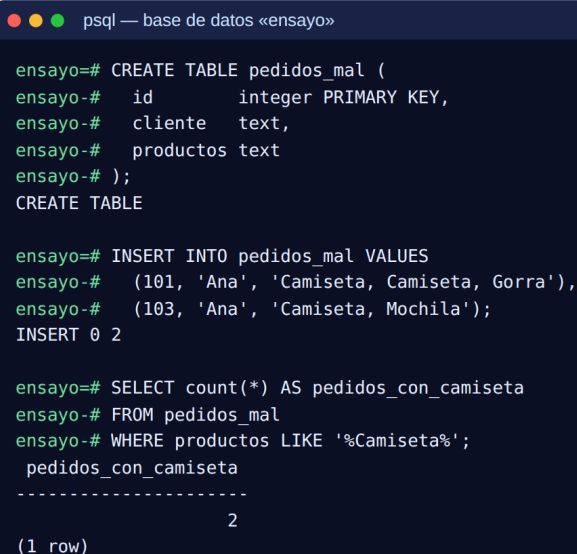
```

CREATE TABLE pedidos_mal (
  id          integer PRIMARY KEY,
  cliente     text,
  productos   text
);

INSERT INTO pedidos_mal VALUES
  (101, 'Ana', 'Camiseta, Camiseta, Gorra'),
  (103, 'Ana', 'Camiseta, Mochila');

SELECT count(*) AS pedidos_con_camiseta
FROM pedidos_mal
WHERE productos LIKE '%Camiseta%';

```



```

psql — base de datos «ensayo»

ensayo=# CREATE TABLE pedidos_mal (
ensayo-#   id          integer PRIMARY KEY,
ensayo-#   cliente     text,
ensayo-#   productos   text
ensayo-# );
ensayo-# CREATE TABLE

ensayo=# INSERT INTO pedidos_mal VALUES
ensayo-#   (101, 'Ana', 'Camiseta, Camiseta, Gorra'),
ensayo-#   (103, 'Ana', 'Camiseta, Mochila');
ensayo-# INSERT 0 2

ensayo=# SELECT count(*) AS pedidos_con_camiseta
ensayo-# FROM pedidos_mal
ensayo-# WHERE productos LIKE '%Camiseta%';
pedidos_con_camiseta
-----
                2
(1 row)

```

Figura 3.3-b. Captura real: con una lista dentro de una celda es imposible contar bien.

Qué ves en la imagen. En esos dos pedidos se han vendido 3 camisetas (2 en el pedido 101 y 1 en el 103), pero la consulta devuelve **2**: cuenta pedidos que contienen la palabra, no unidades. No hay forma sencilla de saber cuántas se vendieron, y `LIKE '%Camiseta%'` también encontraría «Camiseta térmica» por error.

¿Cómo funciona?

La solución son tres tablas en lugar de dos:

1. Una tabla para cada entidad: `pedidos` y `productos`.
2. Una tabla intermedia, `lineas_pedido`, con una fila por cada producto de cada pedido.
3. La intermedia tiene dos claves extranjeras: `pedido_id` y `producto_id`. Juntas forman su **clave primaria compuesta** (de dos columnas), lo que impide que un mismo producto aparezca dos veces en el mismo pedido (si se compra dos veces, se sube la cantidad).

4. Los datos propios de la relación (cantidad y precio de venta) son columnas de la intermedia, porque pertenecen al **par** pedido-producto y no a ninguno de los dos por separado.

Una relación muchos a muchos se convierte así en **dos relaciones uno a muchos**.

Paso a paso: construir las tres tablas de la tienda

1. Crea **productos** (no depende de ninguna otra tabla).
2. Crea **pedidos**, con la clave extranjera **cliente_id** hacia **clientes**, que ya existe.
3. Crea **lineas_pedido**, con dos claves extranjeras y la clave primaria compuesta. Va la última porque apunta a las dos anteriores.
4. Inserta datos en el mismo orden: primero productos y pedidos, y al final las líneas.
5. Comprueba con consultas que los datos se leen bien.

Código y resultado

```
CREATE TABLE productos (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre      text NOT NULL,
  precio_eur  numeric(10, 2) NOT NULL CHECK (precio_eur >= 0),
  stock       integer NOT NULL DEFAULT 0 CHECK (stock >= 0),
  activo      boolean NOT NULL DEFAULT true
);

CREATE TABLE pedidos (
  id          integer GENERATED ALWAYS AS IDENTITY (START WITH 101) PRIMARY KEY,
  cliente_id  integer NOT NULL REFERENCES clientes (id),
  fecha       date NOT NULL DEFAULT current_date,
  estado      text NOT NULL DEFAULT 'pendiente'
              CHECK (estado IN ('pendiente', 'enviado', 'entregado',
                                'cancelado'))
);

CREATE TABLE lineas_pedido (
  pedido_id   integer NOT NULL REFERENCES pedidos (id) ON DELETE CASCADE,
  producto_id integer NOT NULL REFERENCES productos (id),
  cantidad    integer NOT NULL CHECK (cantidad > 0),
  precio_venta numeric(10, 2) NOT NULL CHECK (precio_venta >= 0),
  PRIMARY KEY (pedido_id, producto_id)
);
```



```

psql — base de datos «tienda»

tienda=# CREATE TABLE productos (
tienda-# id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
tienda-# nombre text NOT NULL,
tienda-# precio_eur numeric(10, 2) NOT NULL CHECK (precio_eur >= 0),
tienda-# stock integer NOT NULL DEFAULT 0 CHECK (stock >= 0),
tienda-# activo boolean NOT NULL DEFAULT true
tienda-# );
CREATE TABLE

tienda=# CREATE TABLE pedidos (
tienda-# id integer GENERATED ALWAYS AS IDENTITY (START WITH 101) PRIMARY KEY,
tienda-# cliente_id integer NOT NULL REFERENCES clientes (id),
tienda-# fecha date NOT NULL DEFAULT current_date,
tienda-# estado text NOT NULL DEFAULT 'pendiente'
tienda-# CHECK (estado IN ('pendiente', 'enviado', 'entregado', 'cancelado'))
tienda-# );
CREATE TABLE

tienda=# CREATE TABLE lineas_pedido (
tienda-# pedido_id integer NOT NULL REFERENCES pedidos (id) ON DELETE CASCADE,
tienda-# producto_id integer NOT NULL REFERENCES productos (id),
tienda-# cantidad integer NOT NULL CHECK (cantidad > 0),
tienda-# precio_venta numeric(10, 2) NOT NULL CHECK (precio_venta >= 0),
tienda-# PRIMARY KEY (pedido_id, producto_id)
tienda-# );
CREATE TABLE

```

Figura 3.3-c. Captura real: creación de `productos`, `pedidos` y `lineas\_pedido`.

Qué ves en la imagen. Tres `CREATE TABLE` seguidos y tres respuestas `CREATE TABLE`. Presta atención a tres detalles: `IDENTITY (START WITH 101)` hace que los pedidos empiecen en 101; `ON DELETE CASCADE` hace que al borrar un pedido se borren sus líneas (punto 3.4); y `PRIMARY KEY (pedido_id, producto_id)` es la clave compuesta.

```

INSERT INTO productos (nombre, precio_eur, stock) VALUES
('Camiseta', 19.95, 12), ('Gorra', 9.90, 0), ('Mochila', 34.50, 5);

INSERT INTO pedidos (cliente_id, fecha, estado) VALUES
(1, '2026-03-02', 'entregado'),
(3, '2026-03-05', 'enviado'),
(1, '2026-03-09', 'pendiente');

INSERT INTO lineas_pedido (pedido_id, producto_id, cantidad, precio_venta)
VALUES
(101, 1, 2, 19.95), (101, 2, 1, 9.90),
(102, 3, 1, 34.50),
(103, 1, 1, 19.95), (103, 3, 1, 34.50);

```

```

psql — base de datos «tienda»

tienda=# INSERT INTO productos (nombre, precio_eur, stock) VALUES
tienda-# ('Camiseta', 19.95, 12), ('Gorra', 9.90, 0), ('Mochila', 34.50, 5);
INSERT 0 3

tienda=# INSERT INTO pedidos (cliente_id, fecha, estado) VALUES
tienda-# (1, '2026-03-02', 'entregado'),
tienda-# (3, '2026-03-05', 'enviado'),
tienda-# (1, '2026-03-09', 'pendiente');
INSERT 0 3

tienda=# INSERT INTO lineas_pedido (pedido_id, producto_id, cantidad, precio_venta) VALUES
tienda-# (101, 1, 2, 19.95), (101, 2, 1, 9.90),
tienda-# (102, 3, 1, 34.50),
tienda-# (103, 1, 1, 19.95), (103, 3, 1, 34.50);
INSERT 0 5

```

Figura 3.3-d. Captura real: inserción de productos, pedidos y líneas.

Qué ves en la imagen. `INSERT 0 3`, `INSERT 0 3` e `INSERT 0 5`: tres productos, tres pedidos y cinco líneas. Si hubiéramos insertado las líneas antes que los pedidos, PostgreSQL habría rechazado cada una porque su clave extranjera apuntaría a un pedido que todavía no existe.

Ahora leemos los datos (`SELECT FROM pedidos;` y `SELECT FROM lineas_pedido;`):

pedidos

tienda · pedidos			
id	cliente_id	fecha	estado
101	1	2026-03-02	entregado
102	3	2026-03-05	enviado
103	1	2026-03-09	pendiente

lineas_pedido (la tabla intermedia)

tienda · lineas_pedido			
pedido_id	producto_id	cantidad	precio_venta
101	1	2	19.95
101	2	1	9.90
102	3	1	34.50
103	1	1	19.95
103	3	1	34.50

Captura real: `SELECT * FROM pedidos;` y `SELECT * FROM lineas_pedido;` (resultado mostrado en forma de tabla).

Figura 3.3-e. Captura real: pedidos y su tabla intermedia, con los datos guardados.

Qué ves en la imagen. A la izquierda, tres pedidos. A la derecha, cinco líneas. El pedido 101 aparece en dos líneas (camiseta y gorra) y el producto 1 (camiseta) aparece en dos pedidos distintos (101 y 103). Eso es exactamente una relación muchos a muchos guardada con tres tablas.

Y ahora sí podemos contar bien las camisetas vendidas, y leer el detalle de un pedido:

```

SELECT sum(cantidad) AS camisetas_vendidas
FROM lineas_pedido
WHERE producto_id = 1;

SELECT p.nombre, l.cantidad, l.precio_venta
FROM lineas_pedido l
JOIN productos p ON p.id = l.producto_id
WHERE l.pedido_id = 101;

```

```

psql — base de datos «tienda»

tienda=# SELECT sum(cantidad) AS camisetas_vendidas
tienda=# FROM lineas_pedido
tienda=# WHERE producto_id = 1;
 camisetas_vendidas
-----
                3
(1 row)

tienda=# SELECT p.nombre, l.cantidad, l.precio_venta
tienda=# FROM lineas_pedido l
tienda=# JOIN productos p ON p.id = l.producto_id
tienda=# WHERE l.pedido_id = 101;
 nombre | cantidad | precio_venta
-----+-----+-----
 Camiseta |         2 |         19.95
  Gorra  |         1 |          9.90
(2 rows)

```

Figura 3.3-f. Captura real: sumar unidades y leer el detalle de un pedido.

Qué ves en la imagen. `sum(cantidad)` suma las unidades de las líneas del producto 1: 3 camisetas, que es la respuesta correcta que la tabla del diseño erróneo no pudo dar. La segunda consulta une las líneas con los productos para mostrar el nombre, la cantidad y el precio al que se vendió cada artículo del pedido 101.

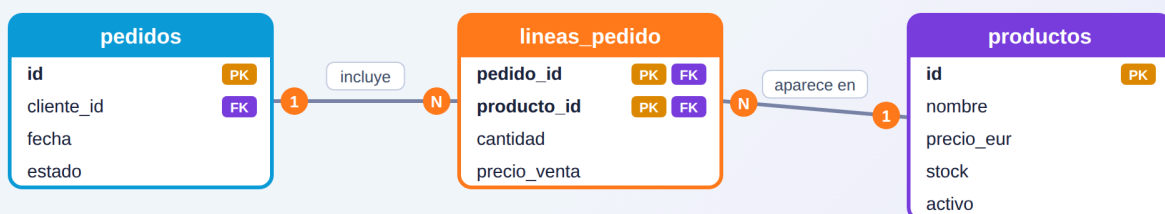


Figura 3.3-g. Diagrama generado leyendo la estructura real de la base de datos.

Qué ves en la imagen. `lineas_pedido` entre `pedidos` y `productos`. Sus dos primeras columnas llevan **PK** y **FK** a la vez: son la clave primaria compuesta y, a la vez, claves extranjeras.

Ejemplo sencillo

El pedido 101 incluye 2 camisetas y 1 gorra: dos filas en `lineas_pedido`, (101, camiseta, 2) y (101, gorra, 1).

Ejemplo real

Cualquier tienda online funciona así: el carrito y el detalle de un pedido son filas de una tabla intermedia. Cuando un producto sube de precio, los pedidos antiguos conservan el `precio_venta` de su día.

Profundizando

Clave compuesta frente a `id` propio. Aquí usamos como clave primaria la pareja (`pedido_id`, `producto_id`). Algunos equipos prefieren añadir una columna `id` a la intermedia y declarar la pareja como `UNIQUE`. Ambas opciones son válidas; la compuesta evita una columna innecesaria, y la del `id` facilita que otras tablas apunten a una línea concreta.

PostgreSQL no crea índices en las claves extranjeras. Crea uno automáticamente para cada clave primaria y cada `UNIQUE`, pero no para las columnas que simplemente referencian a otra tabla. Si vas a buscar a menudo los pedidos de un cliente, conviene crearlo tú:

```
\d pedidos

CREATE INDEX idx_pedidos_cliente ON pedidos (cliente_id);

\d pedidos
```

```

psql — base de datos «tienda»

tienda=# \d pedidos
                                Table "public.pedidos"
  Column  | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
 id       | integer |           | not null | generated always as identity
 cliente_id | integer |           | not null |
 fecha    | date   |           | not null | CURRENT_DATE
 estado   | text   |           | not null | 'pendiente'::text
Indexes:
    "pedidos_pkey" PRIMARY KEY, btree (id)
Check constraints:
    "pedidos_estado_check" CHECK (estado = ANY (ARRAY['pendiente'::text, 'enviado'::text, 'entregado'::text, 'cancelado'::text]))
Foreign-key constraints:
    "pedidos_cliente_id_fkey" FOREIGN KEY (cliente_id) REFERENCES clientes(id)
Referenced by:
    TABLE "lineas_pedido" CONSTRAINT "lineas_pedido_pedido_id_fkey" FOREIGN KEY (pedido_id) REFERENCES pedidos(id) ON DELETE CASCADE

tienda=# CREATE INDEX idx_pedidos_cliente ON pedidos (cliente_id);
CREATE INDEX

tienda=# \d pedidos
                                Table "public.pedidos"
  Column  | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
 id       | integer |           | not null | generated always as identity
 cliente_id | integer |           | not null |
 fecha    | date   |           | not null | CURRENT_DATE
 estado   | text   |           | not null | 'pendiente'::text
Indexes:
    "pedidos_pkey" PRIMARY KEY, btree (id)
    "idx_pedidos_cliente" btree (cliente_id)
Check constraints:
    "pedidos_estado_check" CHECK (estado = ANY (ARRAY['pendiente'::text, 'enviado'::text, 'entregado'::text, 'cancelado'::text]))
Foreign-key constraints:
    "pedidos_cliente_id_fkey" FOREIGN KEY (cliente_id) REFERENCES clientes(id)
Referenced by:
    TABLE "lineas_pedido" CONSTRAINT "lineas_pedido_pedido_id_fkey" FOREIGN KEY (pedido_id) REFERENCES pedidos(id) ON DELETE CASCADE

```

Figura 3.3-h. Captura real: antes y después de crear el índice sobre la clave extranjera `cliente\_id`.

Qué ves en la imagen. El primer `\d pedidos` solo lista el índice de la clave primaria (`pedidos_pkey`). Tras `CREATE INDEX`, el segundo `\d pedidos` muestra además `idx_pedidos_cliente`. En la Parte 18 verás cuánto acelera las consultas.

Datos propios de la relación. Si una columna describe a la pareja (cantidad, nota, fecha de matrícula), va en la intermedia. Si describe a una sola de las entidades (el nombre del producto), va en la entidad.

Ejercicio

1. En una escuela, un profesor imparte muchas asignaturas y una asignatura la imparten varios profesores. Escribe las tres tablas (con columnas mínimas) y la clave primaria de la intermedia.
2. ¿Qué dos datos de una tienda pertenecen al «par» pedido-producto y no a ninguno de los dos?
3. ¿Por qué (`pedido_id, producto_id`) es una buena clave primaria para `lineas_pedido`?

Solución

1. Con la clave compuesta en la intermedia:

```
CREATE TABLE profesores (id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
nombre text NOT NULL);
CREATE TABLE asignaturas (id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
titulo text NOT NULL);
CREATE TABLE imparte (
  profesor_id integer REFERENCES profesores (id),
  asignatura_id integer REFERENCES asignaturas (id),
  PRIMARY KEY (profesor_id, asignatura_id)
);
```

1. La cantidad y el precio de venta.
2. Porque la pareja identifica cada línea de forma única y evita que el mismo producto se repita dentro del mismo pedido.

Error habitual

Meter una lista dentro de una celda («Camiseta, Gorra») o crear columnas repetidas (`producto1`, `producto2` ...). Son los síntomas de que falta una tabla intermedia.

```
-- Mal: una columna por producto
CREATE TABLE pedidos_mal2 (id integer PRIMARY KEY, producto1 text, producto2
text, producto3 text);
```

Buena práctica

Cuando detectes una relación muchos a muchos, crea la tabla intermedia desde el principio, aunque de momento solo tenga dos columnas: casi siempre acaba necesitando datos propios.

Comprobación

Para cada relación de tu diseño pregunta «¿es muchos a muchos?». Si la respuesta es sí y no hay tabla intermedia, falta una. Después comprueba con una consulta que puedes contar y sumar sobre la intermedia, algo imposible en un diseño con listas.

3.4 Claves primarias y claves extranjeras

¿Qué es?

Ya conoces los dos tipos de clave (punto 1.7 de la Parte 1). Aquí aprendes **qué columna elegir como clave primaria** y **cómo se comporta una clave extranjera** cuando alguien intenta borrar o modificar datos.

Para la **clave primaria** hay tres estrategias habituales:

Estrategia	Qué es	Ejemplo
Natural	Un dato real que ya identifica la fila	DNI de una persona, matrícula de un coche
Artificial numérica (identidad)	Un número que el gestor genera por ti: 1, 2, 3...	<code>id</code> de un cliente
UUID	Un identificador largo y aleatorio, único en el mundo	<code>f538b797-fb1b-4e16-be0a-858e3207a1b5</code>

¿Para qué sirve?

Para elegir una forma de identificar cada fila que sea estable (no cambie), única y simple, y para que las claves extranjeras mantengan la coherencia entre tablas.

¿Por qué lo necesito?

Una mala clave primaria obliga a modificar muchas tablas cuando cambia. Si usas el email como clave primaria y un cliente cambia de correo, hay que actualizar todos sus pedidos.

¿Cómo funciona?

Identidad numérica. Es la opción más habitual: `id integer GENERATED ALWAYS AS IDENTITY`. PostgreSQL asigna el siguiente número cada vez que insertas una fila. No tienes que indicarlo en el `INSERT`; de hecho, el gestor lo rechaza si lo intentas (por eso se llama «ALWAYS», «siempre»).

Los números pueden tener «huecos». El número se reserva antes de comprobar si la fila es válida. Si la inserción falla, ese número no se reutiliza.

UUID. Útil cuando los identificadores pueden generarse fuera de la base de datos (por ejemplo, en una app móvil sin conexión) o cuando no quieres que sean adivinables. Son más largos que un número y ocupan más espacio. PostgreSQL los genera con `gen_random_uuid()`.

Clave natural. Sirve cuando el dato es de verdad único e inmutable, pero los datos reales cambian. Por eso la mayoría de proyectos usan una clave artificial y dejan el dato real como columna `UNIQUE`.

Claves extranjeras y borrados. Al intentar borrar una fila a la que otras apuntan, hay varios comportamientos posibles:

Opción	Qué ocurre al borrar la fila «padre»
<code>RESTRICT</code> / <code>NO ACTION</code> (por defecto)	Se rechaza el borrado
<code>ON DELETE CASCADE</code>	Se borran también las filas «hijas»
<code>ON DELETE SET NULL</code>	La columna de las hijas se queda en NULL (si admite NULL)

Paso a paso: probar identidad, huecos y protección

1. **Inserta un cliente sin indicar el `id`** y pide a PostgreSQL que lo devuelva con `RETURNING id`. Verás qué número recibió.
2. **Provoca un error a propósito** (un email repetido). La fila no se guarda, pero el número se gasta.
3. **Inserta otro cliente correcto**: comprobarás que el número saltó.
4. **Intenta forzar un `id` a mano**: PostgreSQL lo rechaza.
5. **Prueba la clave extranjera**: crea un pedido de un cliente inexistente y borra un cliente con pedidos.
6. **Prueba los tres comportamientos de borrado** en tablas de ensayo.

Código y resultado: identidad y huecos

```

INSERT INTO clientes (nombre, email)
VALUES ('Pedro Gil', 'pedro@ejemplo.com')
RETURNING id, nombre;

INSERT INTO clientes (nombre, email)
VALUES ('Otra Ana', 'ana@ejemplo.com');

INSERT INTO clientes (nombre, email)
VALUES ('Sara León', 'sara@ejemplo.com')
RETURNING id, nombre;
```



```

psql — base de datos «tienda»

tienda=# INSERT INTO clientes (nombre, email)
tienda=# VALUES ('Pedro Gil', 'pedro@ejemplo.com')
tienda=# RETURNING id, nombre;
 id | nombre
----+-----
  4 | Pedro Gil
(1 row)

INSERT 0 1

tienda=# INSERT INTO clientes (nombre, email)
tienda=# VALUES ('Otra Ana', 'ana@ejemplo.com');
ERROR:  duplicate key value violates unique constraint "clientes_email_key"
DETAIL:  Key (email)=(ana@ejemplo.com) already exists.

tienda=# INSERT INTO clientes (nombre, email)
tienda=# VALUES ('Sara León', 'sara@ejemplo.com')
tienda=# RETURNING id, nombre;
 id | nombre
----+-----
  6 | Sara León
(1 row)

INSERT 0 1

```

Figura 3.4-a. Captura real: identidad automática y «huecos» en los números.

Qué ves en la imagen. Se inserta a Pedro y recibe el `id` 4 (los tres clientes anteriores usaron 1, 2 y 3). Después se intenta insertar un cliente con un email repetido: falla, pero **gasta el número 5**. Al insertar a Sara, recibe el 6. Los números no son consecutivos, y es lo normal. `RETURNING id, nombre` pide a PostgreSQL que devuelva esas columnas de la fila recién insertada; es la forma estándar de averiguar qué `id` se ha asignado.

```

INSERT INTO clientes (id, nombre, email)
VALUES (50, 'Intruso', 'intruso@ejemplo.com');

```

```

psql — base de datos «tienda»

tienda=# INSERT INTO clientes (id, nombre, email)
tienda=# VALUES (50, 'Intruso', 'intruso@ejemplo.com');
ERROR:  cannot insert a non-DEFAULT value into column "id"
DETAIL:  Column "id" is an identity column defined as GENERATED ALWAYS.
HINT:  Use OVERRIDING SYSTEM VALUE to override.

```

Figura 3.4-b. Captura real: PostgreSQL rechaza que indiques tú el `id`.

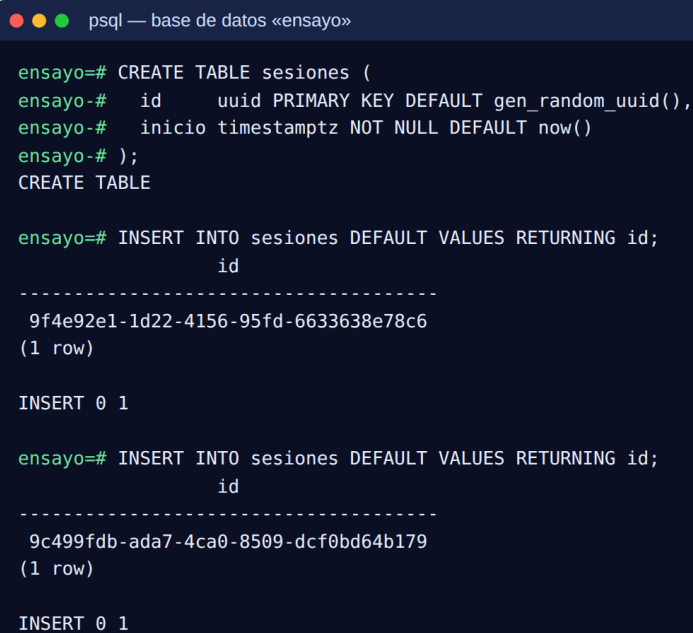
Qué ves en la imagen. El error explica el motivo («`id` es una columna de identidad definida como `GENERATED ALWAYS`») y da una pista (`HINT`): existe `OVERRIDING SYSTEM VALUE` para forzarlo, algo que solo se usa en migraciones de datos. La alternativa `GENERATED BY DEFAULT AS IDENTITY` permite indicar el `id` a mano; es más flexible pero menos segura.

Código y resultado: UUID

```
CREATE TABLE sesiones (
  id      uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  inicio timestampz NOT NULL DEFAULT now()
);

INSERT INTO sesiones DEFAULT VALUES RETURNING id;

INSERT INTO sesiones DEFAULT VALUES RETURNING id;
```



```
psql — base de datos «ensayo»

ensayo=# CREATE TABLE sesiones (
ensayo-#   id      uuid PRIMARY KEY DEFAULT gen_random_uuid(),
ensayo-#   inicio timestampz NOT NULL DEFAULT now()
ensayo-# );
CREATE TABLE

ensayo=# INSERT INTO sesiones DEFAULT VALUES RETURNING id;
          id
-----
9f4e92e1-1d22-4156-95fd-6633638e78c6
(1 row)

INSERT 0 1

ensayo=# INSERT INTO sesiones DEFAULT VALUES RETURNING id;
          id
-----
9c499fdb-ada7-4ca0-8509-dcf0bd64b179
(1 row)

INSERT 0 1
```

Figura 3.4-c. Captura real: una tabla con clave UUID generada por PostgreSQL.

Qué ves en la imagen. La columna `id uuid PRIMARY KEY DEFAULT gen_random_uuid()` se rellena sola, igual que una identidad numérica, pero con un valor largo y aleatorio. Dos inserciones seguidas dan dos UUID totalmente distintos. (Son aleatorios: los de la captura son los que salieron al generar el manual, los tuyos serán otros.)

Código y resultado: las claves extranjeras protegen los datos

```
INSERT INTO pedidos (cliente_id) VALUES (9);

DELETE FROM clientes WHERE id = 1;
```



```
psql — base de datos «tienda»

tienda=# INSERT INTO pedidos (cliente_id) VALUES (9);
ERROR: insert or update on table "pedidos" violates foreign key constraint "pedidos_cliente_id_fkey"
DETAIL: Key (cliente_id)=(9) is not present in table "clientes".

tienda=# DELETE FROM clientes WHERE id = 1;
ERROR: update or delete on table "clientes" violates foreign key constraint "pedidos_cliente_id_fkey" on table "pedidos"
DETAIL: Key (id)=(1) is still referenced from table "pedidos".
```

Figura 3.4-d. Captura real: la clave extranjera rechaza dos operaciones que dejarían datos incoherentes.

Qué ves en la imagen. Dos intentos rechazados. En el primero se crea un pedido para el cliente 9, que no existe: error `is not present in table "clientes"`. En el segundo se intenta borrar al cliente 1, que todavía tiene pedidos: error `is still referenced from table "pedidos"` («sigue referenciado»). La base de datos no permite que los datos se vuelvan incoherentes.

Código y resultado: comportamientos al borrar

```
CREATE TABLE padres (id integer PRIMARY KEY);

CREATE TABLE hijos_cascade (padre_id integer REFERENCES padres (id) ON DELETE CASCADE);

CREATE TABLE hijos_setnull (padre_id integer REFERENCES padres (id) ON DELETE SET NULL);

CREATE TABLE hijos_restrict (padre_id integer REFERENCES padres (id) ON DELETE RESTRICT);

INSERT INTO padres VALUES (1), (2), (3);
INSERT INTO hijos_cascade VALUES (1);
INSERT INTO hijos_setnull VALUES (2);
INSERT INTO hijos_restrict VALUES (3);

DELETE FROM padres WHERE id = 1;

DELETE FROM padres WHERE id = 2;

DELETE FROM padres WHERE id = 3;

SELECT (SELECT count(*) FROM hijos_cascade) AS filas_en_cascade,
       (SELECT count(*) FROM hijos_setnull WHERE padre_id IS NULL) AS
setnull_con_null,
       (SELECT count(*) FROM padres) AS padres_que_quedan;
```

```

psql — base de datos «ensayo»

ensayo=# CREATE TABLE padres (id integer PRIMARY KEY);
CREATE TABLE

ensayo=# CREATE TABLE hijos_cascade (padre_id integer REFERENCES padres (id) ON DELETE CASCADE);
CREATE TABLE

ensayo=# CREATE TABLE hijos_setnull (padre_id integer REFERENCES padres (id) ON DELETE SET NULL);
CREATE TABLE

ensayo=# CREATE TABLE hijos_restrict (padre_id integer REFERENCES padres (id) ON DELETE RESTRICT);
CREATE TABLE

ensayo=# INSERT INTO padres VALUES (1), (2), (3);
ensayo=# INSERT INTO hijos_cascade VALUES (1);
ensayo=# INSERT INTO hijos_setnull VALUES (2);
ensayo=# INSERT INTO hijos_restrict VALUES (3);
INSERT 0 3
INSERT 0 1
INSERT 0 1
INSERT 0 1

ensayo=# DELETE FROM padres WHERE id = 1;
DELETE 1

ensayo=# DELETE FROM padres WHERE id = 2;
DELETE 1

ensayo=# DELETE FROM padres WHERE id = 3;
ERROR:  update or delete on table "padres" violates foreign key constraint "hijos_restrict_padre_id_fkey" on table "hijos_restrict"
DETAIL:  Key (id)=(3) is still referenced from table "hijos_restrict".

ensayo=# SELECT (SELECT count(*) FROM hijos_cascade) AS filas_en_cascade,
ensayo=#          (SELECT count(*) FROM hijos_setnull WHERE padre_id IS NULL) AS setnull_con_null,
ensayo=#          (SELECT count(*) FROM padres) AS padres_que_quedan;
 filas_en_cascade | setnull_con_null | padres_que_quedan
-----+-----+-----
(1 row)          0 | 1 | 1

```

Figura 3.4-e. Captura real: `CASCADE`, `SET NULL` y `RESTRICT` en acción, con tablas de ensayo.

Qué ves en la imagen. Hay tres padres (1, 2 y 3) y un hijo de cada tipo apuntando a uno. Al borrar el padre 1 se borra también su hijo (**CASCADE**): la tabla `hijos_cascade` queda con 0 filas. Al borrar el padre 2, el hijo se queda sin padre (**SET NULL**): 1 fila con NULL. Al borrar el padre 3 se rechaza el borrado (**RESTRICT**) y el padre sigue ahí. La última consulta lo resume: 0, 1 y 1 padre que queda.

Ejemplo sencillo

Un cliente tiene `id` 1. El pedido 101 tiene `cliente_id` 1. Si intentas borrar al cliente 1 mientras tenga pedidos, PostgreSQL lo rechaza.

Ejemplo real

En una tienda, `líneas_pedido` se declara con **ON DELETE CASCADE** hacia `pedidos`: al eliminar un pedido, sus líneas desaparecen con él. Hacia `productos` se deja el comportamiento por defecto: no se puede borrar un producto que ya se ha vendido.

Profundizando

Elegir el comportamiento al borrar. Hazte la pregunta: «¿tiene sentido la fila hija sin el padre?». Una línea de pedido sin pedido no tiene sentido (**CASCADE**). Un empleado sin departamento sí puede tener sentido (**SET NULL**). Una factura sin cliente no (**RESTRICT**).

Secuencias. Debajo de una columna de identidad hay una **secuencia**, un contador que PostgreSQL mantiene aparte. Por eso el contador no «retrocede» al fallar una inserción ni al borrar filas. Se explica en la Parte 5.

Qué clave elegir. Regla práctica: identidad numérica para la mayoría de tablas; UUID cuando el identificador se genera fuera de la base de datos o no debe adivinarse; clave natural solo si es verdaderamente inmutable.

Ejercicio

1. ¿Qué estrategia usarías para una tabla de `vehiculos` con matrícula? Razona.
2. Hay tres clientes con `id` 1, 2 y 3. Se intenta insertar uno nuevo que falla por un dato inválido, y después otro correcto. ¿Qué `id` recibe el segundo?
3. Escribe la clave extranjera para que, al borrar un pedido, se borren sus líneas.

Solución

1. Un `id` artificial como clave primaria y `matricula text NOT NULL UNIQUE`: las matrículas pueden corregirse. Usarla como clave natural solo si estás seguro de que nunca cambia.
2. El 5, no el 4: el 4 se gastó en el intento fallido.
3. `pedido_id integer NOT NULL REFERENCES pedidos (id) ON DELETE CASCADE`.

Error habitual

Usar un dato que cambia (email, nombre, teléfono) como clave primaria, o dar por hecho que los `id` automáticos son consecutivos y usarlos para contar filas.

```
-- Mal: contar filas con el mayor id (los huecos lo falsean)
SELECT max(id) FROM clientes;
-- Bien: contar filas de verdad
SELECT count(*) FROM clientes;
```

Buena práctica

Usa una clave artificial como clave primaria y deja los datos reales importantes como `UNIQUE`. Decide siempre qué debe pasar al borrar: rechazar, borrar en cascada o dejar vacío.

Comprobación

Para cada tabla: ¿qué columna es la clave primaria y por qué nunca cambiará? Para cada clave extranjera: ¿qué debe pasar si se borra la fila a la que apunta? Comprueba la respuesta con `\d tabla`, en el apartado «Foreign-key constraints».

3.5 Restricciones: únicos, obligatorios, nulos y comprobaciones

¿Qué es?

Una **restricción** (en inglés, *constraint*) es una regla que la base de datos vigila siempre. Si alguien intenta guardar un dato que la incumple, PostgreSQL **rechaza la operación** y devuelve un error. Las principales:

Restricción	Qué impone	Ejemplo
<code>NOT NULL</code>	La columna no puede quedarse sin valor	El nombre de un cliente
<code>UNIQUE</code>	No puede haber dos filas con el mismo valor	El email de un cliente
<code>PRIMARY KEY</code>	Obligatoria y única a la vez; identifica la fila	El <code>id</code>
<code>FOREIGN KEY</code> (<code>REFERENCES</code>)	El valor debe existir en otra tabla	El <code>cliente_id</code> de un pedido
<code>CHECK</code>	El valor debe cumplir una condición que tú escribes	El precio no es negativo
<code>DEFAULT</code>	No es una prohibición: es un valor por defecto	El estado «pendiente»

¿Para qué sirve?

Para que las reglas del negocio no dependan de que cada programador las recuerde. Da igual que el dato venga de la web, de la app móvil o de una automatización: todas pasan por la misma puerta, y la base de datos es el último guardián.

¿Por qué lo necesito?

Los programas tienen errores, los usuarios escriben cosas inesperadas y las automatizaciones se rompen. Si la regla vive solo en la aplicación, algún día alguien guardará un precio negativo o un cliente sin nombre. Si vive en la base de datos, nunca podrá pasar.

¿Cómo funciona?

Las restricciones se declaran al crear la tabla (o después, con `ALTER TABLE`). Cada vez que insertas o modificas una fila, PostgreSQL las comprueba todas. Si alguna falla, la fila **no se guarda** y recibes un mensaje con dos partes:

- **ERROR:** qué regla se incumplió y el nombre de la restricción.
- **DETAIL:** el detalle: qué valor o qué fila lo causó.

Si no pones nombre a una restricción, PostgreSQL le da uno automático con un patrón:

`tabla_columna_key` para `UNIQUE`, `tabla_columna_check` para `CHECK`, `tabla_columna_fkey` para claves extranjeras y `tabla_pkey` para la clave primaria. Esos nombres aparecen en los errores.

Paso a paso: comprobar que cada regla funciona

1. **Revisa la estructura** de la tabla con `\d` para ver qué restricciones tiene.
2. **Intenta romper cada regla a propósito**, una por una, con una sentencia mínima.
3. **Lee el mensaje de error**: la primera línea dice qué regla saltó; `DETAIL` muestra la fila o el valor.
4. **Confirma que no se guardó nada** con un `SELECT`.
5. Si necesitas añadir una regla a una tabla que ya existe, usa `ALTER TABLE ... ADD CONSTRAINT`. Falla si ya hay datos que la incumplen: hay que corregirlos antes.

Código y resultado: cinco reglas, cinco rechazos

```
INSERT INTO clientes (nombre, email)
VALUES (NULL, 'x@ejemplo.com');

INSERT INTO clientes (nombre, email)
VALUES ('Otra Ana', 'ana@ejemplo.com');

INSERT INTO productos (nombre, precio_eur)
VALUES ('Regalo', -5);

INSERT INTO lineas_pedido
VALUES (101, 1, 0, 19.95);

UPDATE pedidos SET estado = 'casi-enviado' WHERE id = 101;
```

```

psql — base de datos «tienda»

tienda=# INSERT INTO clientes (nombre, email)
tienda-# VALUES (NULL, 'x@ejemplo.com');
ERROR: null value in column "nombre" of relation "clientes" violates not-null constraint
DETAIL: Failing row contains (7, null, x@ejemplo.com, null, 2026-10-03 08:11:44.149674+00).

tienda=# INSERT INTO clientes (nombre, email)
tienda-# VALUES ('Otra Ana', 'ana@ejemplo.com');
ERROR: duplicate key value violates unique constraint "clientes_email_key"
DETAIL: Key (email)=(ana@ejemplo.com) already exists.

tienda=# INSERT INTO productos (nombre, precio_eur)
tienda-# VALUES ('Regalo', -5);
ERROR: new row for relation "productos" violates check constraint "productos_precio_eur_check"
DETAIL: Failing row contains (4, Regalo, -5.00, 0, t).

tienda=# INSERT INTO lineas_pedido
tienda-# VALUES (101, 1, 0, 19.95);
ERROR: new row for relation "lineas_pedido" violates check constraint "lineas_pedido_cantidad_check"
DETAIL: Failing row contains (101, 1, 0, 19.95).

tienda=# UPDATE pedidos SET estado = 'casi-enviado' WHERE id = 101;
ERROR: new row for relation "pedidos" violates check constraint "pedidos_estado_check"
DETAIL: Failing row contains (101, 1, 2026-03-02, casi-enviado).

```

Figura 3.5-a. Captura real: cinco datos incorrectos rechazados por cinco restricciones distintas.

Qué ves en la imagen.

1. **NOT NULL**: cliente sin nombre. El error nombra la columna. En **DETAIL**, la fila rechazada muestra **null** donde faltaba el nombre; el primer número es el **id** que se había reservado (el 7).
2. **UNIQUE**: email repetido. El error nombra la restricción **clientes_email_key** y el valor repetido.
3. **CHECK**: precio de -5. Nombra **productos_precio_eur_check**.
4. **CHECK**: línea con cantidad 0. Nombra **lineas_pedido_cantidad_check**.
5. **CHECK con lista de valores**: un estado inventado, «casi-enviado», es rechazado: solo se admiten los cuatro estados permitidos.

Código y resultado: añadir una restricción a una tabla existente

```

CREATE TABLE articulos (nombre text, precio numeric);

INSERT INTO articulos VALUES ('Lámpara', 25), ('Pegatina', -3);

ALTER TABLE articulos ADD CONSTRAINT precio_no_negativo CHECK (precio >= 0);

DELETE FROM articulos WHERE precio < 0;

ALTER TABLE articulos ADD CONSTRAINT precio_no_negativo CHECK (precio >= 0);

INSERT INTO articulos VALUES ('Imán', -1);

```



```

psql — base de datos «ensayo»

ensayo=# CREATE TABLE articulos (nombre text, precio numeric);
CREATE TABLE

ensayo=# INSERT INTO articulos VALUES ('Lámpara', 25), ('Pegatina', -3);
INSERT 0 2

ensayo=# ALTER TABLE articulos ADD CONSTRAINT precio_no_negativo CHECK (precio >= 0);
ERROR:  check constraint "precio_no_negativo" of relation "articulos" is violated by some row

ensayo=# DELETE FROM articulos WHERE precio < 0;
DELETE 1

ensayo=# ALTER TABLE articulos ADD CONSTRAINT precio_no_negativo CHECK (precio >= 0);
ALTER TABLE

ensayo=# INSERT INTO articulos VALUES ('Imán', -1);
ERROR:  new row for relation "articulos" violates check constraint "precio_no_negativo"
DETAIL:  Failing row contains (Imán, -1).

```

Figura 3.5-b. Captura real: añadir una restricción falla si ya hay datos que la incumplen.

Qué ves en la imagen. La tabla `articulos` se crea sin reglas y se le inserta un precio negativo (−3). Al intentar añadir `CHECK (precio >= 0)`, PostgreSQL responde `is violated by some row`: hay filas que no cumplen la regla. Tras borrar la fila incorrecta, la restricción se añade correctamente (`ALTER TABLE`), y desde entonces un precio negativo (el imán de −1) es rechazado. Ponerle un nombre propio (`precio_no_negativo`) hace los errores más legibles.

```
\d productos
```

```

psql — base de datos «tienda»

tienda=# \d productos
               Table "public.productos"
   Column   | Type          | Collation | Nullable | Default                |
-----+-----+-----+-----+-----+
 id         | integer       |           | not null | generated always as identity |
 nombre     | text          |           | not null |                             |
 precio_eur | numeric(10,2) |           | not null |                             |
 stock      | integer       |           | not null | 0                         |
 activo     | boolean       |           | not null | true                      |
Indexes:
    "productos_pkey" PRIMARY KEY, btree (id)
Check constraints:
    "productos_precio_eur_check" CHECK (precio_eur >= 0::numeric)
    "productos_stock_check" CHECK (stock >= 0)
Referenced by:
    TABLE "lineas_pedido" CONSTRAINT "lineas_pedido_producto_id_fkey" FOREIGN KEY (producto_id) REFERENCES productos(id)

```

Figura 3.5-c. Captura real: todas las restricciones de `productos` vistas con `ld`.

Qué ves en la imagen. En «Check constraints» aparecen las dos condiciones de `productos` (precio y stock no negativos). El apartado «Referenced by» muestra que `lineas_pedido` apunta a esta tabla. `\d` es la forma más rápida de comprobar las reglas de una tabla.

Ejemplo sencillo

Un `CHECK` de precio mayor o igual que cero se escribe `precio_eur numeric(10, 2) NOT NULL CHECK (precio_eur >= 0)`. Con eso, un producto con precio negativo no puede existir.

Ejemplo real

En `pedidos`, el estado solo admite cuatro valores, y por defecto es «pendiente»:

```
estado text NOT NULL DEFAULT 'pendiente'
CHECK (estado IN ('pendiente', 'enviado', 'entregado', 'cancelado'))
```

Profundizando

NULL y UNIQUE. En PostgreSQL, los valores NULL no se consideran iguales entre sí a efectos de `UNIQUE`, así que una columna `UNIQUE` sin `NOT NULL` admite varias filas con NULL. Es útil para datos opcionales pero únicos (código de barras).

Restricciones entre columnas. Un `CHECK` puede comparar varias columnas de la misma fila: `CHECK (fecha_devolucion >= fecha_salida)`, como en la biblioteca del punto 3.7.

Restricciones con nombre. Puedes nombrarlas: `CONSTRAINT precio_no_negativo CHECK (precio >= 0)`. Los errores serán más claros y podrás eliminarlas por su nombre (`ALTER TABLE ... DROP CONSTRAINT`).

Tipos enumerados. Para listas cerradas de valores (como el estado), PostgreSQL ofrece los tipos `ENUM`. Un `CHECK ... IN (...)` es más fácil de modificar; los `ENUM` son más estrictos. Lo trataremos en la Parte 6.

Ejercicio

1. Escribe la restricción que impide un `stock` negativo en una tabla ya creada.
2. ¿Qué restricción usarías para que no haya dos productos con el mismo código de barras, permitiendo que algunos no tengan código?
3. Lee este error y explica qué ha pasado: `ERROR: duplicate key value violates unique constraint "clientes_email_key", DETAIL: Key (email)=(ana@ejemplo.com) already exists.`

Solución

1. `ALTER TABLE productos ADD CONSTRAINT stock_no_negativo CHECK (stock >= 0);`
2. `UNIQUE` sobre la columna `codigo_barras`, sin `NOT NULL`: varios NULL conviven sin problema.
3. Se intentó guardar un cliente con el email `ana@ejemplo.com`, pero ya existe otro con ese email y la columna es única.

Error habitual

Poner las reglas solo en la aplicación («ya lo valida el formulario»). Basta con que otro programa escriba en la base de datos, o que alguien se salte la validación, para que entren datos incorrectos.

```
-- Sin restricción: la base de datos acepta cualquier cosa
CREATE TABLE ofertas (porcentaje integer);
INSERT INTO ofertas VALUES (-500);      -- se guarda
-- Con restricción: se rechaza
CREATE TABLE ofertas_ok (porcentaje integer CHECK (porcentaje BETWEEN 0 AND 100));
```

Buena práctica

Pon en la base de datos todas las reglas que **siempre** deben cumplirse. La aplicación puede validarlas también, para dar mensajes más amables, pero la base de datos es la que nunca falla.

Comprobación

Para cada columna pregunta: ¿puede faltar? (`NOT NULL`), ¿puede repetirse? (`UNIQUE`), ¿hay valores imposibles? (`CHECK`), ¿hay un valor habitual? (`DEFAULT`). Después intenta romper cada regla a propósito y confirma que PostgreSQL la rechaza.

3.6 Diagramas entidad-relación

¿Qué es?

Un **diagrama entidad-relación** (abreviado **ER**) es un dibujo del diseño: cada entidad es una caja con sus atributos, y las líneas entre cajas son las relaciones, con su cardinalidad. Es el plano de la base de datos, igual que el plano de una casa se dibuja antes de construirla.

¿Para qué sirve?

Para ver el diseño completo de un vistazo, discutirlo con otras personas y detectar problemas (falta una relación, hay una tabla suelta, una relación está al revés) antes de escribir SQL. También sirve de documentación.

¿Por qué lo necesito?

Una base de datos con diez tablas es muy difícil de entender leyendo solo código. Con el diagrama, alguien nuevo entiende el sistema en cinco minutos.

¿Cómo funciona?

La notación que usamos en este manual:

1. Cada **caja** es una tabla; su cabecera de color lleva el nombre.
2. Cada **línea de texto** es una columna.
3. **PK** marca la clave primaria y **FK** la clave extranjera.
4. Una **línea** une dos tablas; los **números** junto a cada extremo indican la cardinalidad (1 o N).
5. Una etiqueta sobre la línea (por ejemplo «hace») nombra la relación con un verbo.



- 1 · Cada caja es una tabla
- 2 · La cabecera de color es su nombre
- 3 · Cada línea de texto es una columna
- 4 · **PK** clave primaria
- 5 · **FK** clave extranjera
- 6 · La línea es la relación; los números indican cuántos

Figura 3.6-a. Cómo leer un diagrama: las seis piezas de la notación.

Qué ves en la imagen. Un diagrama mínimo con su leyenda: cada parte numerada, de la tabla a la cardinalidad.

Paso a paso: dibujar un diagrama

1. **Dibuja una caja por cada entidad**, con su nombre en la cabecera.
2. **Escribe las columnas** dentro de cada caja. Marca la clave primaria (PK).
3. **Une con líneas** las cajas relacionadas.

4. **Pon la cardinalidad** en cada extremo, leyendo la relación en los dos sentidos.
5. **Marca las claves extranjeras (FK)** en las tablas del lado «N».
6. **Revisa:** ¿hay tablas sin ninguna línea? ¿Hay relaciones N a N sin tabla intermedia?

Código y resultado: generar el diagrama desde la base de datos real

La mejor forma de no equivocarse es **leer la estructura de la propia base de datos**. PostgreSQL guarda la descripción de todas las tablas en su **catálogo** (tablas internas del sistema). Esta consulta lista todas las claves extranjeras de `tienda`:

```
SELECT conrelid::regclass AS tabla,
       conname           AS restriccion,
       pg_get_constraintdef(oid) AS definicion
FROM pg_constraint
WHERE contype = 'f'
      AND connamespace = 'public'::regnamespace
ORDER BY 1, 2;
```



```
psql — base de datos «tienda»

tienda=# SELECT conrelid::regclass AS tabla,
tienda=#         conname           AS restriccion,
tienda=#         pg_get_constraintdef(oid) AS definicion
tienda=# FROM pg_constraint
tienda=# WHERE contype = 'f'
tienda=# AND connamespace = 'public'::regnamespace
tienda=# ORDER BY 1, 2;
 tabla | restriccion | definicion
-----+-----+-----
 pedidos | pedidos_cliente_id_fkey | FOREIGN KEY (cliente_id) REFERENCES clientes(id)
 lineas_pedido | lineas_pedido_pedido_id_fkey | FOREIGN KEY (pedido_id) REFERENCES pedidos(id) ON DELETE CASCADE
 lineas_pedido | lineas_pedido_producto_id_fkey | FOREIGN KEY (producto_id) REFERENCES productos(id)
(3 rows)
```

Figura 3.6-b. Captura real: las claves extranjeras de la base de datos, leídas del catálogo del sistema.

Qué ves en la imagen. Tres claves extranjeras: `pedidos` apunta a `clientes`, y `lineas_pedido` apunta a `pedidos` (con `ON DELETE CASCADE`) y a `productos`. Cada una trae el nombre de la restricción y su definición completa. `pg_constraint` es la tabla del catálogo que guarda las restricciones; `pg_get_constraintdef` convierte cada una a texto legible.

Con esa información (más la de las columnas) se dibuja el diagrama completo:

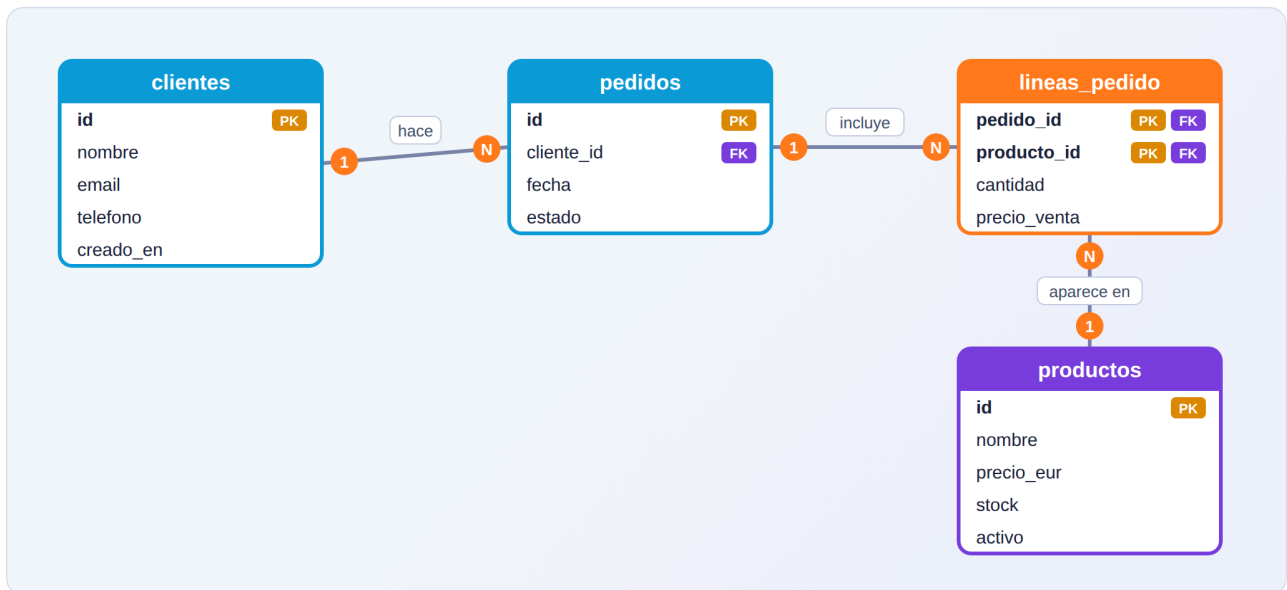


Figura 3.6-c. Diagrama de la tienda generado a partir del catálogo real de PostgreSQL.

Qué ves en la imagen. Las cuatro tablas con sus columnas, sus claves primarias (PK) y sus claves extranjeras (FK). No se ha dibujado a mano: el programa que genera este manual consulta el catálogo y lo dibuja, por eso coincide exactamente con la base de datos.

Ejemplo sencillo

Para «un cliente hace pedidos» dibujas dos cajas, una línea con «1» en el lado de `clientes` y «N» en el de `pedidos`, y la columna `cliente_id` marcada FK en `pedidos`.

Ejemplo real

En un equipo de desarrollo, el diagrama se guarda junto al código y se actualiza cuando cambia el diseño. Es lo primero que se enseña a quien se incorpora al proyecto.

Profundizando

Notación de «pata de gallo». Muchas herramientas dibujan la cardinalidad con símbolos en los extremos de la línea (una línea simple para «uno» y tres trazos como la pata de un ave para «muchos») en lugar de números. Es la misma información con otra apariencia.

Diagramas en texto. Existen lenguajes para describir diagramas con texto, que luego se renderizan. Un ejemplo es la sintaxis `erDiagram` de Mermaid. Esta descripción de la tienda es válida en esa sintaxis (aquí no la hemos renderizado; se muestra solo como ejemplo de la idea):

```

erDiagram
    CLIENTES ||--o{ PEDIDOS : hace
    PEDIDOS ||--o{ LINEAS_PEDIDO : incluye
    PRODUCTOS ||--o{ LINEAS_PEDIDO : aparece_en
  
```

Generarlo desde clientes gráficos. Los clientes gráficos de la Parte 7 suelen poder generar un diagrama a partir de una base de datos existente.

Ejercicio

1. Dibuja (en papel) el diagrama ER de una biblioteca con socios, libros y préstamos.
2. Lee la figura 3.6-c: ¿qué tablas tienen claves extranjeras y cuántas en total?
3. ¿Qué te dice sobre el diseño que **productos** no tenga ninguna clave extranjera?

Solución

1. Tres cajas, **socios**, **libros** y **prestamos**; **prestamos** tiene dos claves extranjeras (hacia **socios** y hacia **libros**). Es el diagrama de la figura 3.7-c.
2. Dos tablas: **pedidos** (una) y **lineas_pedido** (dos). En total, tres.
3. Que **productos** es una entidad independiente: no depende de ninguna otra para existir. Otras tablas la referencian, pero ella a ninguna.

Error habitual

Dibujar el diagrama una vez al principio y no actualizarlo. Un diagrama desactualizado es peor que no tener ninguno.

Buena práctica

Genera el diagrama a partir de la base de datos real siempre que sea posible, para que nunca difiera de ella.

Comprobación

Con el diagrama, otra persona debería poder responder: ¿cuántas tablas hay?, ¿cómo se relacionan?, ¿qué tabla toco si quiero guardar este dato nuevo? Y debería coincidir con lo que muestra **\d** en cada tabla.

3.7 Del diagrama a las tablas

¿Qué es?

Es el paso final del diseño: traducir el diagrama a las sentencias de SQL que crean las tablas, siguiendo **reglas de conversión** siempre iguales.

¿Para qué sirve?

Para pasar del plano a la construcción sin improvisar. Cualquier persona que siga las reglas llegará a las mismas tablas.

¿Por qué lo necesito?

Es donde se juntan todos los conceptos de la parte. Si dominas la conversión, puedes crear la base de datos de cualquier proyecto a partir de su diagrama.

¿Cómo funciona?

Elemento del diagrama	Qué se hace al crear las tablas
Entidad	Una tabla con una clave primaria
Atributo	Una columna con su tipo; obligatoria si hace falta
Uno a muchos (1 a N)	Una clave extranjera en la tabla del lado «muchos»
Uno a uno (1 a 1)	Una clave extranjera que además es única (o es la clave primaria)
Muchos a muchos (N a N)	Una tabla intermedia con dos claves extranjeras
Regla	Una restricción: NOT NULL, UNIQUE, CHECK...

Ilustración: reglas de conversión.

Figura 3.7-a. Ilustración: reglas para convertir un diagrama en tablas.

Qué ves en la imagen. Cada elemento del diagrama tiene una traducción fija: entidad a tabla; atributo a columna; uno a muchos a clave extranjera en el lado «muchos»; uno a uno a clave extranjera única; muchos a muchos a tabla intermedia; regla a restricción.

Un detalle importante: el **orden de creación**. Una tabla solo puede apuntar a otra que **ya exista**. Se crean primero las tablas independientes (`clientes` , `productos`), después `pedidos` y, por último, `lineas_pedido` .

Paso a paso: del plano a una base de datos funcionando

1. **Escribe todas las sentencias en un archivo** (por ejemplo, `crear_tienda.sql`), en el orden correcto.
2. **Crea la base de datos vacía.**

3. **Ejecuta el archivo** con `psql -f`. Cada sentencia exitosa imprime `CREATE TABLE`.
4. **Comprueba** con `\dt` (lista de tablas) y `\d tabla` (estructura de cada una).
5. **Compara con el diagrama**: mismas tablas, mismas claves, mismas relaciones.

Código y resultado

El archivo `crear_tienda.sql` es el script completo de nuestra tienda. Ya has visto sus cuatro sentencias a lo largo de la parte:

```
CREATE TABLE clientes (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre      text NOT NULL,
  email       text NOT NULL UNIQUE,
  telefono    text,
  creado_en   timestampz NOT NULL DEFAULT now()
);

CREATE TABLE productos (
  id          integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre      text NOT NULL,
  precio_eur  numeric(10, 2) NOT NULL CHECK (precio_eur >= 0),
  stock       integer NOT NULL DEFAULT 0 CHECK (stock >= 0),
  activo      boolean NOT NULL DEFAULT true
);

CREATE TABLE pedidos (
  id          integer GENERATED ALWAYS AS IDENTITY (START WITH 101) PRIMARY KEY,
  cliente_id  integer NOT NULL REFERENCES clientes (id),
  fecha       date NOT NULL DEFAULT current_date,
  estado      text NOT NULL DEFAULT 'pendiente'
              CHECK (estado IN ('pendiente', 'enviado', 'entregado',
                                'cancelado'))
);

CREATE TABLE lineas_pedido (
  pedido_id   integer NOT NULL REFERENCES pedidos (id) ON DELETE CASCADE,
  producto_id integer NOT NULL REFERENCES productos (id),
  cantidad    integer NOT NULL CHECK (cantidad > 0),
  precio_venta numeric(10, 2) NOT NULL CHECK (precio_venta >= 0),
  PRIMARY KEY (pedido_id, producto_id)
);
```

Ejecutarlo desde la terminal, sobre una base de datos nueva llamada `tienda_script` (los parámetros `-h`, `-p` y `-U` indican el servidor, el puerto y el usuario; en tu instalación pueden ser otros):

```
head -8 crear_tienda.sql
psql -U postgres -d tienda_script -f crear_tienda.sql
```

```

Terminal

$ head -8 crear_tienda.sql
CREATE TABLE clientes (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre  text NOT NULL,
  email   text NOT NULL UNIQUE,
  telefono text,
  creado_en timestampz NOT NULL DEFAULT now()
);
$ psql -U postgres -d tienda_script -f crear_tienda.sql
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE

```

Figura 3.7-b. Captura real: ejecución del script desde la terminal con `psql -f`.

Qué ves en la imagen. `head -8` muestra las primeras líneas del archivo. `psql -f` ejecuta todo el archivo, y responde con una línea `CREATE TABLE` por cada tabla creada: cuatro en total. Si el archivo contuviera un error, `psql` lo indicaría con la línea y el motivo.

```
\dt
```

```
\d lineas_pedido
```

```

psql — base de datos «tienda_script»

tienda_script=# \dt
          List of relations
Schema |   Name   | Type | Owner
-----+-----+-----+-----
public | clientes | table | postgres
public | lineas_pedido | table | postgres
public | pedidos | table | postgres
public | productos | table | postgres
(4 rows)

tienda_script=# \d lineas_pedido
          Table "public.lineas_pedido"
   Column   |   Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
pedido_id  | integer  |           | not null |
producto_id | integer  |           | not null |
cantidad   | integer  |           | not null |
precio_venta | numeric(10,2) |         | not null |
Indexes:
    "lineas_pedido_pkey" PRIMARY KEY, btree (pedido_id, producto_id)
Check constraints:
    "lineas_pedido_cantidad_check" CHECK (cantidad > 0)
    "lineas_pedido_precio_venta_check" CHECK (precio_venta >= 0::numeric)
Foreign-key constraints:
    "lineas_pedido_pedido_id_fkey" FOREIGN KEY (pedido_id) REFERENCES pedidos(id) ON DELETE CASCADE
    "lineas_pedido_producto_id_fkey" FOREIGN KEY (producto_id) REFERENCES productos(id)

```

Figura 3.7-c. Captura real: las tablas creadas (`\dt`) y la estructura de la intermedia (`\d lineas\_pedido`).

Qué ves en la imagen. `\dt` lista las cuatro tablas. `\d lineas_pedido` muestra la estructura completa de la tabla intermedia: sus cuatro columnas (todas `not null`), la clave primaria compuesta, las dos restricciones `CHECK` y las dos claves extranjeras (con `ON DELETE CASCADE` hacia `pedidos`).

Ejemplo sencillo

Para «un cliente hace muchos pedidos»: se crea `clientes`; se crea `pedidos` con `cliente_id integer NOT NULL REFERENCES clientes (id)`.

Ejemplo real

En un proyecto de verdad, este archivo se guarda en el repositorio y es el primer script que se ejecuta al montar un entorno nuevo (desarrollo, pruebas o producción).

Profundizando

Orden y dependencias. Si creas `pedidos` antes que `clientes`, PostgreSQL responde que la relación `clientes` no existe. El orden correcto es siempre: tablas independientes primero, dependientes después.

Idempotencia. Un script que se puede ejecutar varias veces sin fallar usa `CREATE TABLE IF NOT EXISTS`, o empieza borrando lo anterior con `DROP TABLE IF EXISTS`. En desarrollo es muy cómodo; en producción hay que tener mucho cuidado con los `DROP`.

Migraciones. Los cambios posteriores (añadir una columna, crear una tabla) no se hacen editando el script inicial, sino con scripts nuevos y numerados, que se ejecutan en orden. A esa práctica se le llama migraciones y se explica en la Parte 7.

Ejercicio

Práctica integrada: una biblioteca. Diseña y crea una base de datos para este enunciado: «Una biblioteca presta libros a socios. De cada socio se guarda nombre y email (único); de cada libro, título y autor. Cada préstamo guarda quién se llevó qué libro, la fecha de salida (por defecto, hoy) y la fecha de devolución (vacía hasta que lo devuelva). Un libro no puede estar prestado a dos socios a la vez».

Solución

Entidades: socios, libros y préstamos. Un socio tiene muchos préstamos y un libro aparece en muchos préstamos, así que `prestamos` es una tabla con dos claves extranjeras. Este script se ha ejecutado en PostgreSQL:

```

CREATE TABLE socios (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre  text NOT NULL,
  email   text NOT NULL UNIQUE
);

CREATE TABLE libros (
  id      integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  titulo  text NOT NULL,
  autor   text NOT NULL
);

CREATE TABLE prestamos (
  id                integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  socio_id          integer NOT NULL REFERENCES socios (id),
  libro_id          integer NOT NULL REFERENCES libros (id),
  fecha_salida      date NOT NULL DEFAULT current_date,
  fecha_devolucion  date,
  CHECK (fecha_devolucion IS NULL OR fecha_devolucion >= fecha_salida)
);

CREATE UNIQUE INDEX un_prestamo_abierto_por_libro
  ON prestamos (libro_id) WHERE fecha_devolucion IS NULL;

```

La última sentencia es un índice único **parcial**: solo mira las filas con `fecha_devolucion IS NULL`, es decir, los préstamos sin devolver. Entre esas filas no permite repetir `libro_id`: un libro no puede tener dos préstamos abiertos, pero sí muchos ya devueltos. Los índices parciales se explican en la Parte 6.



Figura 3.7-d. Diagrama generado a partir de la base de datos real `biblioteca`.

Qué ves en la imagen. `prestamos` en el centro, con dos claves extranjeras hacia `socios` y `libros`: una relación muchos a muchos resuelta con una tabla intermedia que además guarda datos propios (las fechas).

```

INSERT INTO socios (nombre, email) VALUES ('Ana García', 'ana@ejemplo.com');

INSERT INTO libros (titulo, autor) VALUES ('Cien años de soledad', 'García
Márquez');

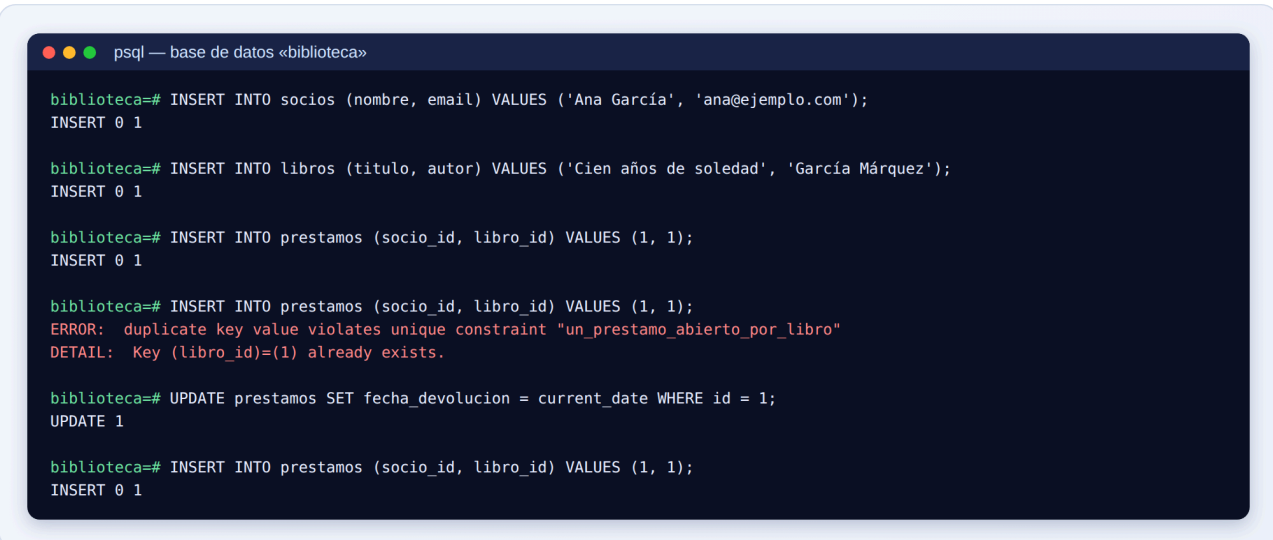
INSERT INTO prestamos (socio_id, libro_id) VALUES (1, 1);

INSERT INTO prestamos (socio_id, libro_id) VALUES (1, 1);

UPDATE prestamos SET fecha_devolucion = current_date WHERE id = 1;

INSERT INTO prestamos (socio_id, libro_id) VALUES (1, 1);

```



```

psql — base de datos «biblioteca»

biblioteca=# INSERT INTO socios (nombre, email) VALUES ('Ana García', 'ana@ejemplo.com');
INSERT 0 1

biblioteca=# INSERT INTO libros (titulo, autor) VALUES ('Cien años de soledad', 'García Márquez');
INSERT 0 1

biblioteca=# INSERT INTO prestamos (socio_id, libro_id) VALUES (1, 1);
INSERT 0 1

biblioteca=# INSERT INTO prestamos (socio_id, libro_id) VALUES (1, 1);
ERROR: duplicate key value violates unique constraint "un_prestamo_abierto_por_libro"
DETAIL: Key (libro_id)=(1) already exists.

biblioteca=# UPDATE prestamos SET fecha_devolucion = current_date WHERE id = 1;
UPDATE 1

biblioteca=# INSERT INTO prestamos (socio_id, libro_id) VALUES (1, 1);
INSERT 0 1

```

Figura 3.7-e. Captura real: la regla «un libro, un préstamo abierto» en acción.

Qué ves en la imagen. Se crea un socio y un libro. El primer préstamo se acepta; el segundo, del mismo libro sin que se haya devuelto el primero, es rechazado por el índice parcial. Después se devuelve el libro (`UPDATE ... SET fecha_devolucion`) y un nuevo préstamo del mismo libro vuelve a ser aceptado: el índice solo vigila los préstamos abiertos.

Error habitual

Crear las tablas en orden incorrecto, o editar el script inicial para cada cambio (y perder el rastro de qué cambió y cuándo).

```

-- Falla: clientes todavía no existe
CREATE TABLE pedidos (id integer PRIMARY KEY, cliente_id integer REFERENCES
clientes (id));

```

Buena práctica

Guarda el script de creación en un archivo junto al proyecto. Así puedes reconstruir la base de datos desde cero cuando quieras, y el archivo sirve de documentación.

Comprobación

Ejecuta el script en una base de datos vacía, y después `\dt` y `\d` en cada tabla. Debe coincidir con tu diagrama: mismas tablas, mismas claves, mismas relaciones.

3.8 Convenciones: nombres, fechas y borrado lógico

¿Qué es?

Las **convenciones** son acuerdos de estilo que hacen que una base de datos sea predecible: cómo se llaman las cosas, qué columnas «de control» lleva cada tabla y cómo se «borran» datos sin perderlos.

¿Para qué sirve?

Para que cualquiera pueda adivinar cómo se llama una columna sin mirarla y para que los datos no se pierdan por accidente.

¿Por qué lo necesito?

Cuando un proyecto crece hay decenas de tablas. Si unas usan `clienteId`, otras `id_cliente` y otras `CLIENTE`, cada consulta es una adivinanza. Y si «borrar» elimina los datos para siempre, un error humano puede costar días de trabajo.

¿Cómo funciona?

Nombres. Las reglas que usamos en todo el manual:

- Tablas en **plural** y en minúsculas: `clientes`, `pedidos`.
- Columnas en **minúsculas**, con **guiones bajos**, sin tildes ni espacios: `fecha_nacimiento`, `precio_eur`.
- Clave primaria: `id`.
- Clave extranjera: nombre de la tabla a la que apunta, en singular, más `_id`: `cliente_id`.
- Tablas intermedias: un nombre que diga lo que representan (`lineas_pedido`) o los dos nombres juntos (`profesor_asignatura`).

(¿Por qué sin mayúsculas ni tildes? PostgreSQL convierte a minúsculas los nombres sin comillas. Si usas mayúsculas, tendrías que escribir comillas dobles cada vez.)

Columnas de control. Casi todas las tablas conviene que tengan `creado_en` (cuándo se creó la fila) y, a menudo, `actualizado_en` (cuándo se modificó por última vez; se mantiene con un *trigger* o desde la aplicación, y los triggers se explican en la Parte 5).

Borrado lógico. En lugar de borrar con `DELETE`, se «marca» la fila como borrada con una columna `borrado_en`, que está en NULL mientras la fila esté viva y se rellena con la fecha cuando se «borra». Así el dato se puede recuperar y las referencias no se rompen. La contrapartida: **todas las consultas deben filtrar** las filas borradas.

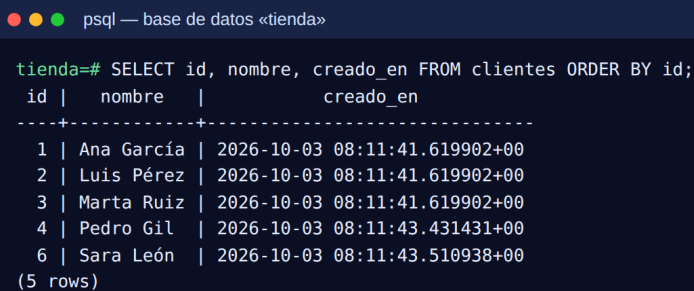
Paso a paso: columna de control y borrado lógico

1. **Observa** `creado_en`: la rellena PostgreSQL gracias a `DEFAULT now()`.
2. **Añade la columna** `borrado_en` a `clientes` con `ALTER TABLE ... ADD COLUMN`.

3. «Borra» un cliente con `UPDATE` : no se elimina, solo se marca.
4. Consulta todos los clientes y comprueba que la fila sigue ahí.
5. Consulta solo los vivos con `WHERE borrado_en IS NULL` .
6. Evita el olvido del filtro creando una **vista** (una consulta guardada con nombre) que ya lo incluya.

Código y resultado

```
SELECT id, nombre, creado_en FROM clientes ORDER BY id;
```



```
psql — base de datos «tienda»

tienda=# SELECT id, nombre, creado_en FROM clientes ORDER BY id;
 id | nombre |      creado_en      |
----+-----+-----+
  1 | Ana García | 2026-10-03 08:11:41.619902+00 |
  2 | Luis Pérez | 2026-10-03 08:11:41.619902+00 |
  3 | Marta Ruiz | 2026-10-03 08:11:41.619902+00 |
  4 | Pedro Gil | 2026-10-03 08:11:43.431431+00 |
  6 | Sara León | 2026-10-03 08:11:43.510938+00 |
(5 rows)
```

Figura 3.8-a. Captura real: la columna `creado\_en`, rellenada automáticamente.

Qué ves en la imagen. Ninguna inserción indicó la fecha: la rellenó PostgreSQL con `DEFAULT now()` . Las fechas y horas son las del momento en que se generó el manual.

```
ALTER TABLE clientes ADD COLUMN borrado_en timestamptz;

UPDATE clientes SET borrado_en = now() WHERE id = 2;

SELECT id, nombre, borrado_en FROM clientes ORDER BY id;

SELECT id, nombre FROM clientes WHERE borrado_en IS NULL ORDER BY id;
```



```
psql — base de datos «tienda»

tienda=# ALTER TABLE clientes ADD COLUMN borrado_en timestampz;
ALTER TABLE

tienda=# UPDATE clientes SET borrado_en = now() WHERE id = 2;
UPDATE 1

tienda=# SELECT id, nombre, borrado_en FROM clientes ORDER BY id;
 id | nombre |      borrado_en
-----+-----+-----
  1 | Ana García |
  2 | Luis Pérez | 2026-10-03 08:11:45.574828+00
  3 | Marta Ruiz |
  4 | Pedro Gil |
  6 | Sara León |
(5 rows)

tienda=# SELECT id, nombre FROM clientes WHERE borrado_en IS NULL ORDER BY id;
 id | nombre
-----+-----
  1 | Ana García
  3 | Marta Ruiz
  4 | Pedro Gil
  6 | Sara León
(4 rows)
```

Figura 3.8-b. Captura real: borrado lógico con una columna `borrado\_en`.

Qué ves en la imagen. `ALTER TABLE ... ADD COLUMN` añade la columna (las filas existentes quedan con NULL). `UPDATE 1` indica que se modificó una fila: Luis queda marcado. La primera consulta lista a todos, con la marca de Luis. La segunda, con `WHERE borrado_en IS NULL`, muestra solo los clientes vivos: Luis ya no aparece, pero sus datos siguen en la tabla.

```
CREATE VIEW clientes_activos AS
SELECT id, nombre, email, telefono
FROM clientes
WHERE borrado_en IS NULL;

SELECT * FROM clientes_activos ORDER BY id;
```

```
psql — base de datos «tienda»

tienda=# CREATE VIEW clientes_activos AS
tienda=# SELECT id, nombre, email, telefono
tienda=# FROM clientes
tienda=# WHERE borrado_en IS NULL;
CREATE VIEW

tienda=# SELECT * FROM clientes_activos ORDER BY id;
 id | nombre | email | telefono
-----+-----+-----+-----
  1 | Ana García | ana@ejemplo.com | 600111222
  3 | Marta Ruiz | marta@ejemplo.com | 600333444
  4 | Pedro Gil | pedro@ejemplo.com |
  6 | Sara León | sara@ejemplo.com |
(4 rows)
```

Figura 3.8-c. Captura real: una vista que oculta los clientes borrados.

Qué ves en la imagen. `CREATE VIEW` guarda la consulta con el filtro bajo el nombre `clientes_activos`. A partir de aquí se consulta como si fuera una tabla, y nadie se olvida del `WHERE borrado_en IS NULL`. Las vistas se explican en profundidad en la Parte 5.

Ejemplo sencillo

La tabla `clientes` tiene la columna `creado_en`, que se rellena sola al insertar. No hace falta que ninguna aplicación se acuerde de ponerla.

Ejemplo real

Una tienda no borra nunca a un cliente que ya ha comprado: se marca con `borrado_en` y desaparece de las listas, pero sus pedidos, facturas y estadísticas siguen intactos.

Profundizando

Protección de datos. El borrado lógico conserva los datos, lo cual puede entrar en conflicto con la normativa de protección de datos (RGPD): si una persona pide que se eliminen sus datos, a veces hay que eliminarlos de verdad o anonimizarlos. Se trata en la Parte 17.

Borrado lógico y `UNIQUE`. Si un cliente «borrado» conserva su email, nadie podrá registrarse de nuevo con él. La solución habitual es un índice único parcial: `CREATE UNIQUE INDEX ... ON clientes (email) WHERE borrado_en IS NULL`.

Nombres de restricciones e índices. Dar a índices y restricciones nombres coherentes (`idx_tabla_columna`, `un_...`) facilita leer errores y planes de ejecución.

Ejercicio

1. Corrige estos nombres según las convenciones: `Cliente`, `fechaDeNacimiento`, `ID_Pedido`, `Precio €`.
2. Escribe la columna que añadirías a una tabla `facturas` para saber cuándo se creó cada una sin que nadie la escriba.
3. Un compañero hace `SELECT * FROM clientes` y muestra en la web los clientes «borrados». ¿Qué ha olvidado y cómo se evita?

Solución

1. `clientes` (tabla), `fecha_de_nacimiento`, `id` (o `pedido_id` si es clave extranjera), `precio_eur`.
2. `creado_en timestampz NOT NULL DEFAULT now()`.
3. Ha olvidado `WHERE borrado_en IS NULL`. Se evita consultando una vista (`clientes_activos`) que ya incluya el filtro.

Error habitual

Usar el borrado lógico y olvidar el filtro en alguna consulta, de modo que aparecen datos «borrados».

```
-- Mal: incluye clientes borrados
SELECT count(*) FROM clientes;
-- Bien: solo los vivos
SELECT count(*) FROM clientes WHERE borrado_en IS NULL;
```

Buena práctica

Decide las convenciones al empezar el proyecto, escríbelas en una página y aplícalas sin excepciones: es mucho más barato que corregir cientos de nombres más tarde.

Comprobación

Elige tres tablas de tu proyecto. ¿Siguen las mismas convenciones de nombres? ¿Tienen `creado_en`? ¿Qué pasa al «borrar» una fila: se pierde o se puede recuperar?

Resumen de la Parte 3

- Cada entidad es una tabla y cada atributo una columna con su tipo; las decisiones de obligatorio, único y valor por defecto se guardan en la tabla.
- La cardinalidad (1 a 1, 1 a N, N a N) se decide preguntando en los dos sentidos.
- Una relación muchos a muchos se resuelve con una tabla intermedia con dos claves extranjeras y datos propios de la pareja.
- La clave primaria debe ser estable y única: lo habitual es una identidad numérica; los datos reales se dejan como `UNIQUE`.
- Las claves extranjeras protegen la coherencia y definen qué pasa al borrar (`RESTRICT`, `CASCADE`, `SET NULL`).
- Las restricciones (`NOT NULL`, `UNIQUE`, `CHECK`) hacen que PostgreSQL rechace los datos incorrectos.
- Un diagrama ER es el plano del diseño y puede generarse desde el catálogo real.
- La conversión es mecánica: entidad en tabla, atributo en columna, 1 a N en clave extranjera, N a N en intermedia; se crean primero las tablas independientes.
- Las convenciones (nombres, `creado_en`, borrado lógico) hacen la base predecible y segura.

Glosario de la Parte 3

Término	Significado
Cardinalidad	Cuántos elementos de un lado se conectan con cuántos del otro
Tabla intermedia	Tabla que resuelve una relación muchos a muchos
Clave compuesta	Clave primaria formada por dos o más columnas
Identidad	Número automático que genera el gestor para cada fila
UUID	Identificador largo y aleatorio, único en el mundo
Restricción (constraint)	Regla que la base de datos vigila siempre
ON DELETE CASCADE / SET NULL / RESTRICT	Qué ocurre con las filas hijas al borrar la fila padre
Relación reflexiva	Una tabla que se relaciona consigo misma (empleado y jefe)
Catálogo	Tablas internas donde PostgreSQL describe la base de datos
Diagrama ER	Dibujo del diseño: tablas, claves y relaciones
Borrado lógico	Marcar una fila como borrada sin eliminarla
Vista	Consulta guardada con nombre que se usa como una tabla
<code>psql</code>	Cliente de terminal de PostgreSQL
RETURNING	Devuelve datos de la fila recién insertada

Mini examen de la Parte 3

1. ¿Qué dos preguntas haces para decidir la cardinalidad de una relación?
2. ¿En qué tabla va la clave extranjera de una relación uno a muchos?
3. ¿Por qué una relación muchos a muchos necesita una tabla intermedia?
4. Nombra dos datos que pertenecen a `lineas_pedido` y no a `pedidos` ni a `productos`.
5. ¿Por qué se suele preferir una clave artificial a una natural?
6. Si una inserción falla, ¿se reutiliza el número de identidad reservado?
7. ¿Qué diferencia hay entre `NOT NULL` y `UNIQUE`?
8. ¿Qué hace `ON DELETE CASCADE`?

9. ¿Por qué falla `ALTER TABLE ... ADD CONSTRAINT CHECK` en una tabla con datos incorrectos?
10. ¿Qué ventaja tiene el borrado lógico y qué cuidado exige?

Respuestas

1. ¿Cuántos B puede tener un A? y ¿cuántos A puede tener un B?
2. En la tabla del lado «muchos».
3. Porque no se pueden guardar listas dentro de una celda ni columnas repetidas sin límite; la intermedia tiene una fila por cada pareja.
4. La cantidad y el precio de venta.
5. Porque los datos reales pueden cambiar o repetirse; una clave artificial no cambia.
6. No: el número queda «gastado» y la siguiente inserción recibe otro.
7. `NOT NULL` obliga a que haya valor; `UNIQUE` impide que el valor se repita entre filas.
8. Al borrar una fila, borra también las filas de otras tablas que apuntan a ella.
9. Porque PostgreSQL comprueba la regla contra las filas existentes y rechaza añadirla si alguna no la cumple.
10. Los datos se pueden recuperar y no se rompen las referencias; exige filtrar las filas borradas en todas las consultas.

Si has acertado 8 o más, estás listo para la Parte 4. Si no, repasa los puntos de las preguntas falladas.